

GİRİŞ.

Eğitmen

Özgür YILDIRIM



[linkedin.com/in/Ozgur-yldrm/](https://www.linkedin.com/in/Ozgur-yldrm/)

github.com/OzgurYldrm

İzzet YILMAZ



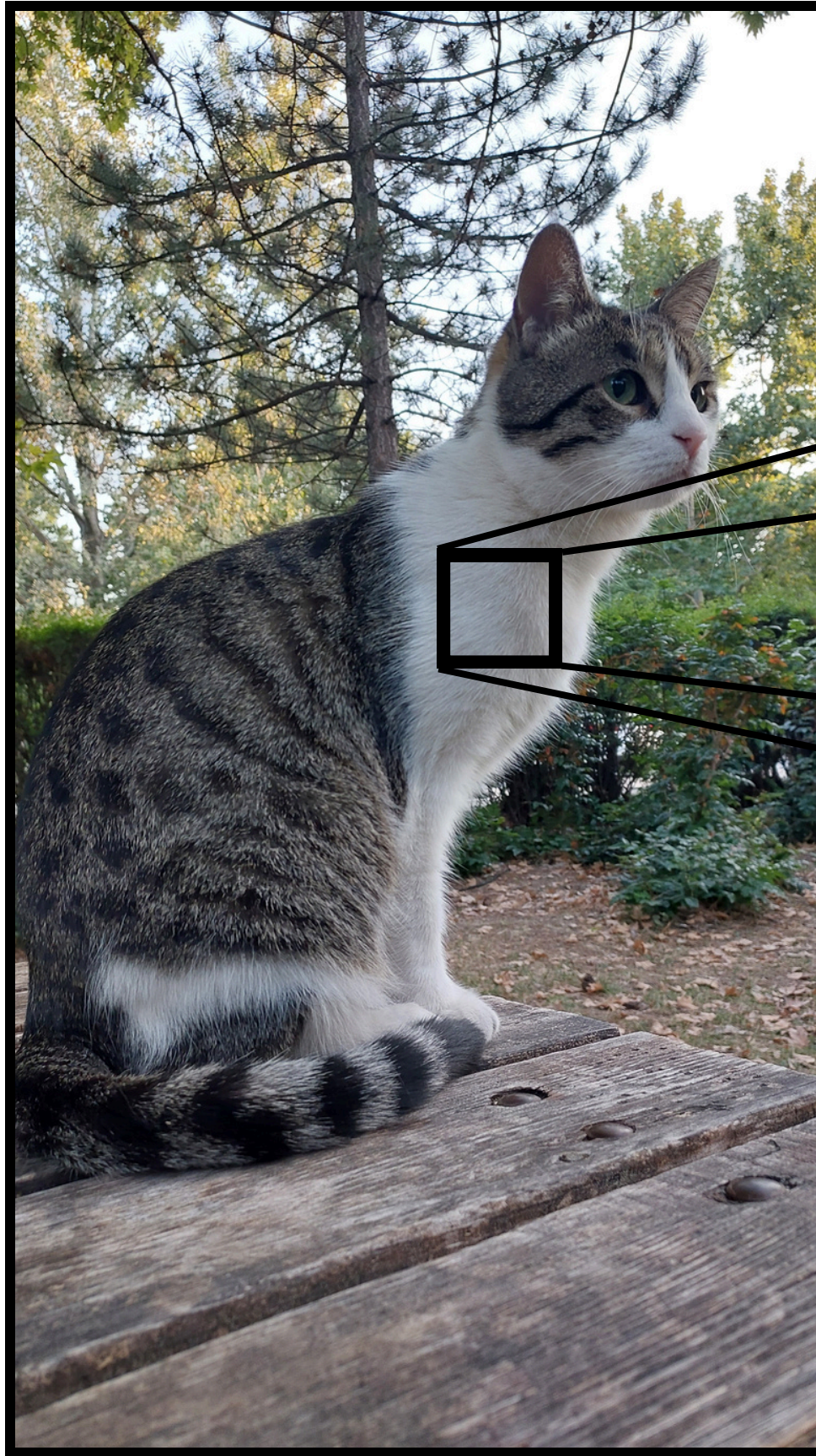
[linkedin.com/in/izzet-yilmazz/](https://www.linkedin.com/in/izzet-yilmazz/)

github.com/izzetyilmazz

Sunum + Notebook + Homework → Github

- **Image Data**
- **Image Operations**
- **Convolution Operation**
- **Convolution Layer**
- **Pooling Layer**
- **Fully Connected Layer**
- **CNN**
- **Transfer Learning**
- **SoTA's**
- **Data Augmentation**

Pixel

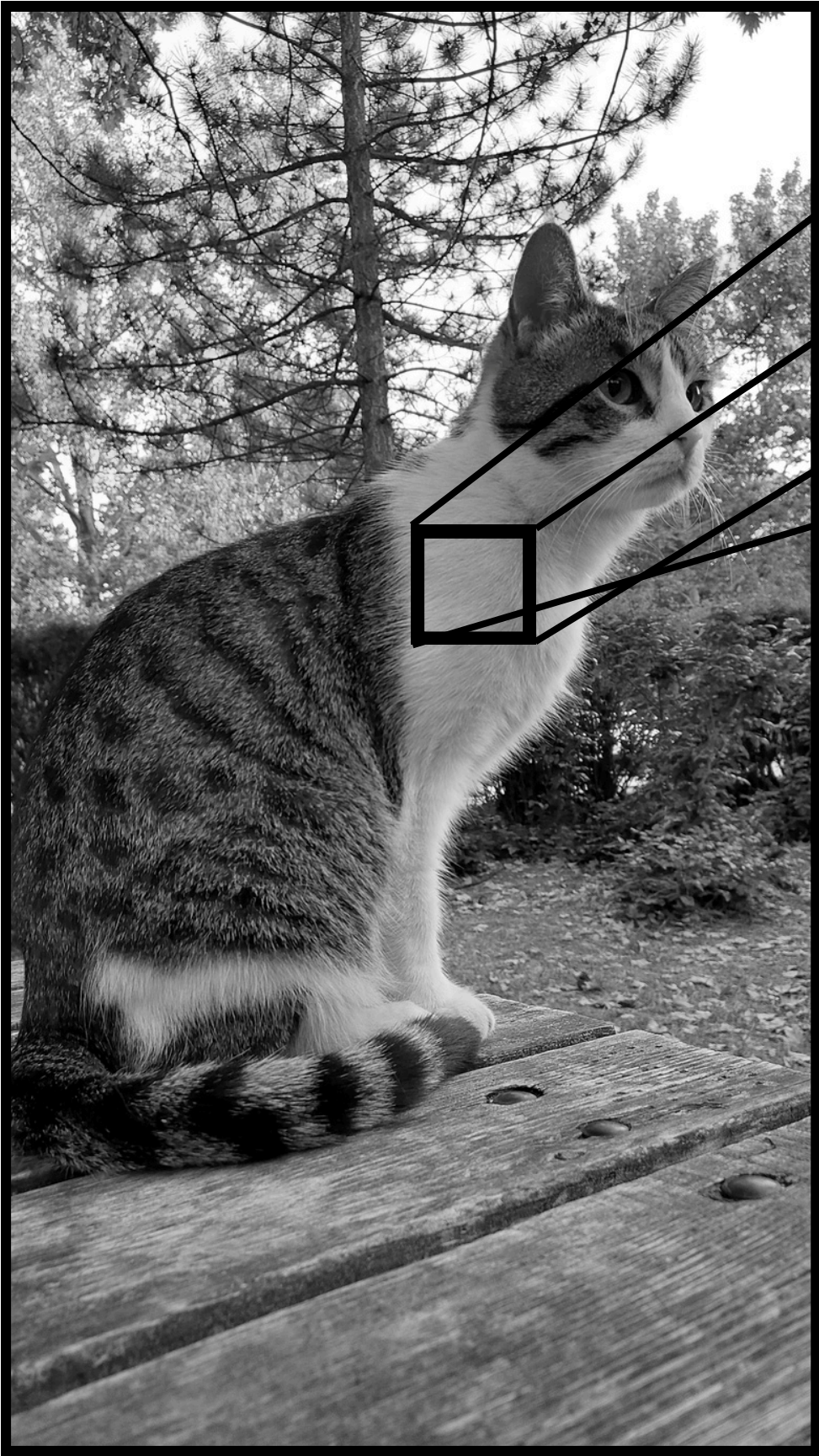


Pixel	Pixel	Pixel
Pixel	Pixel	Pixel
Pixel	Pixel	Pixel
Pixel	Pixel	Pixel

- **Pixel Image data'nın en küçük yapı birimidir**

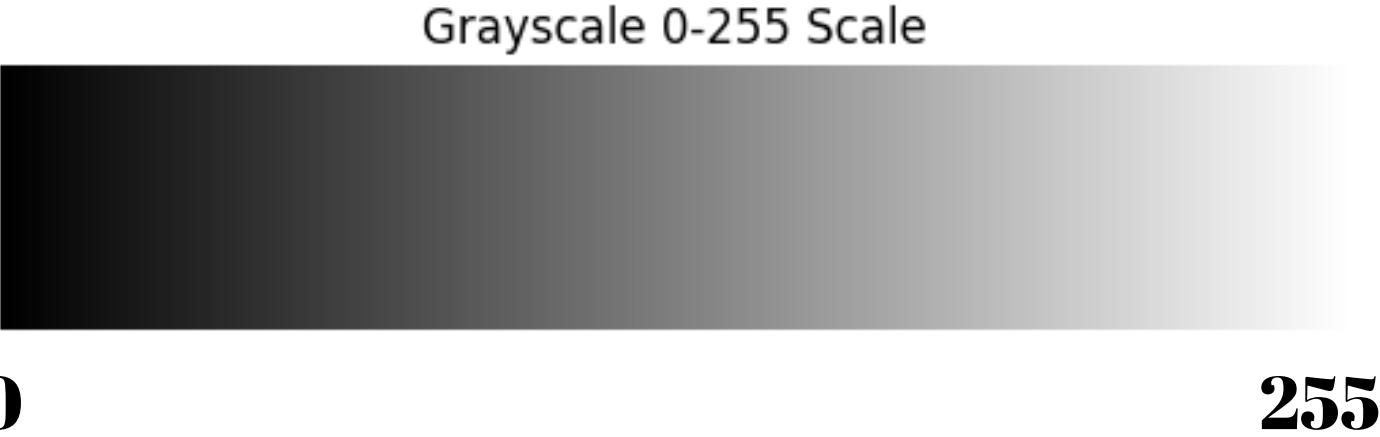
width = 2604
height = 4624> **# pixel = 12040896**

GrayScale

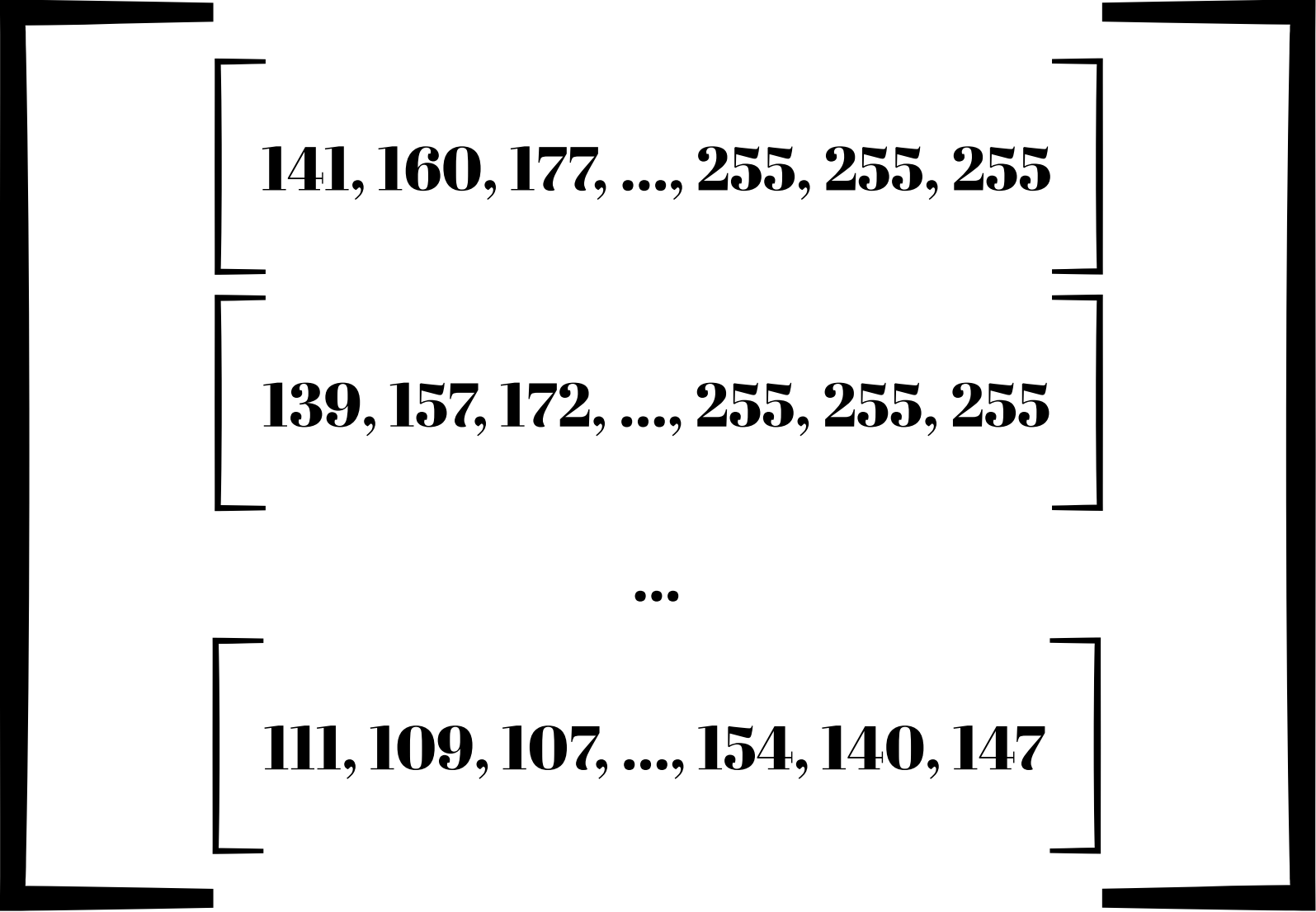


(4624, 2604)

196	199	188
193	178	158
164	144	120

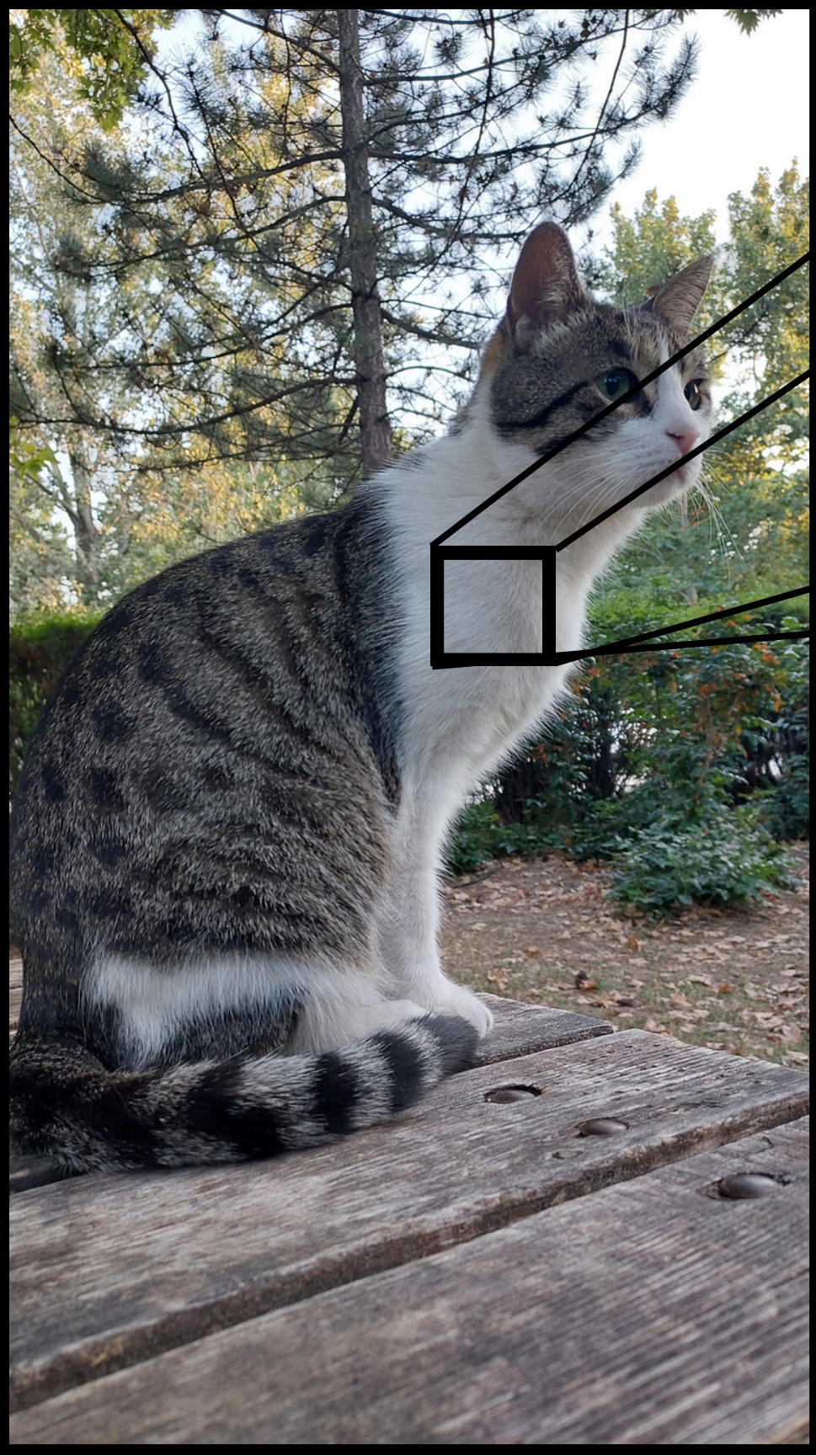


height = 4624

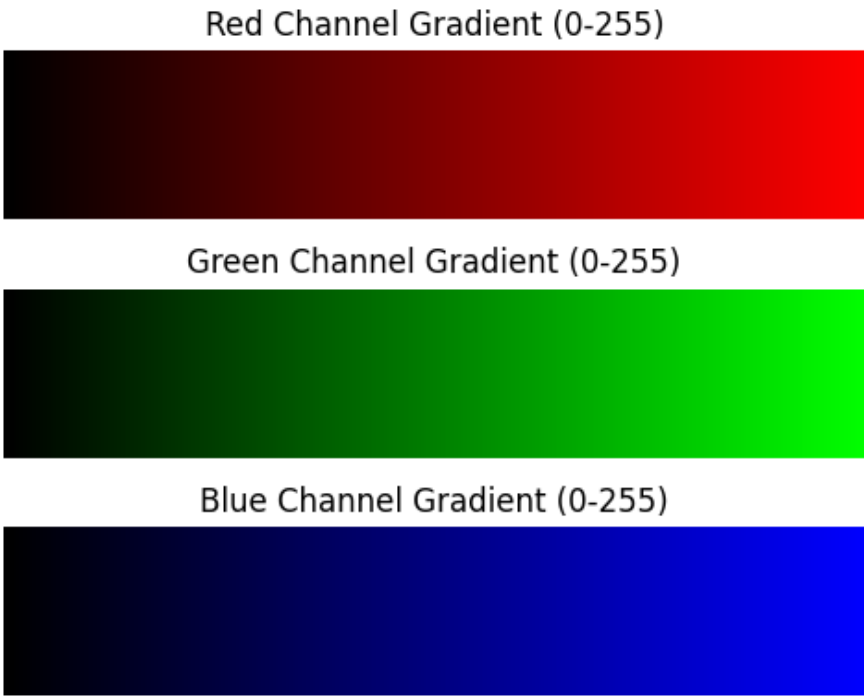


width = 2604

RGB



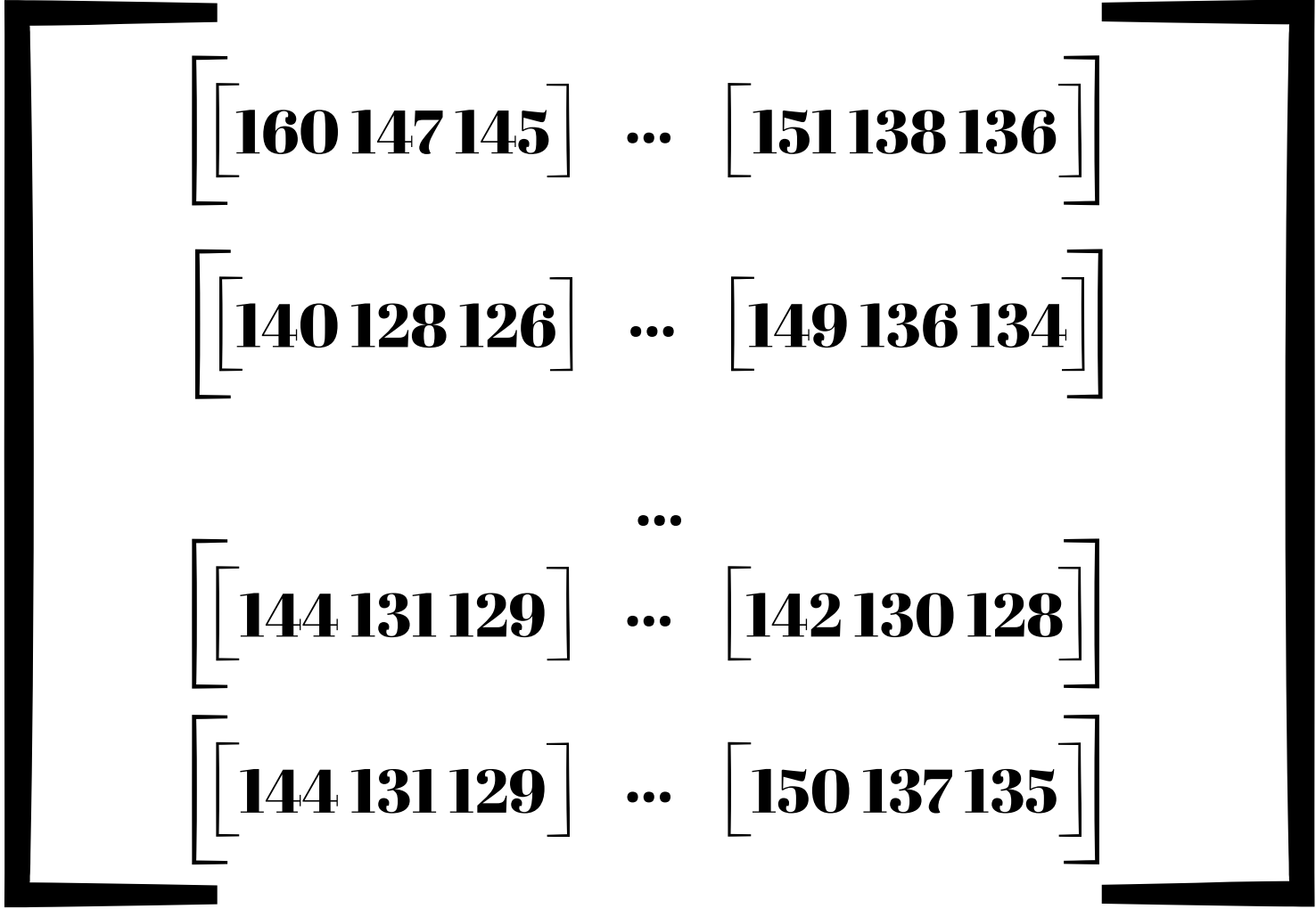
RGB	RGB	RGB
RGB	RGB	RGB
RGB	RGB	RGB
RGB	RGB	RGB



(4624, 2604,3)

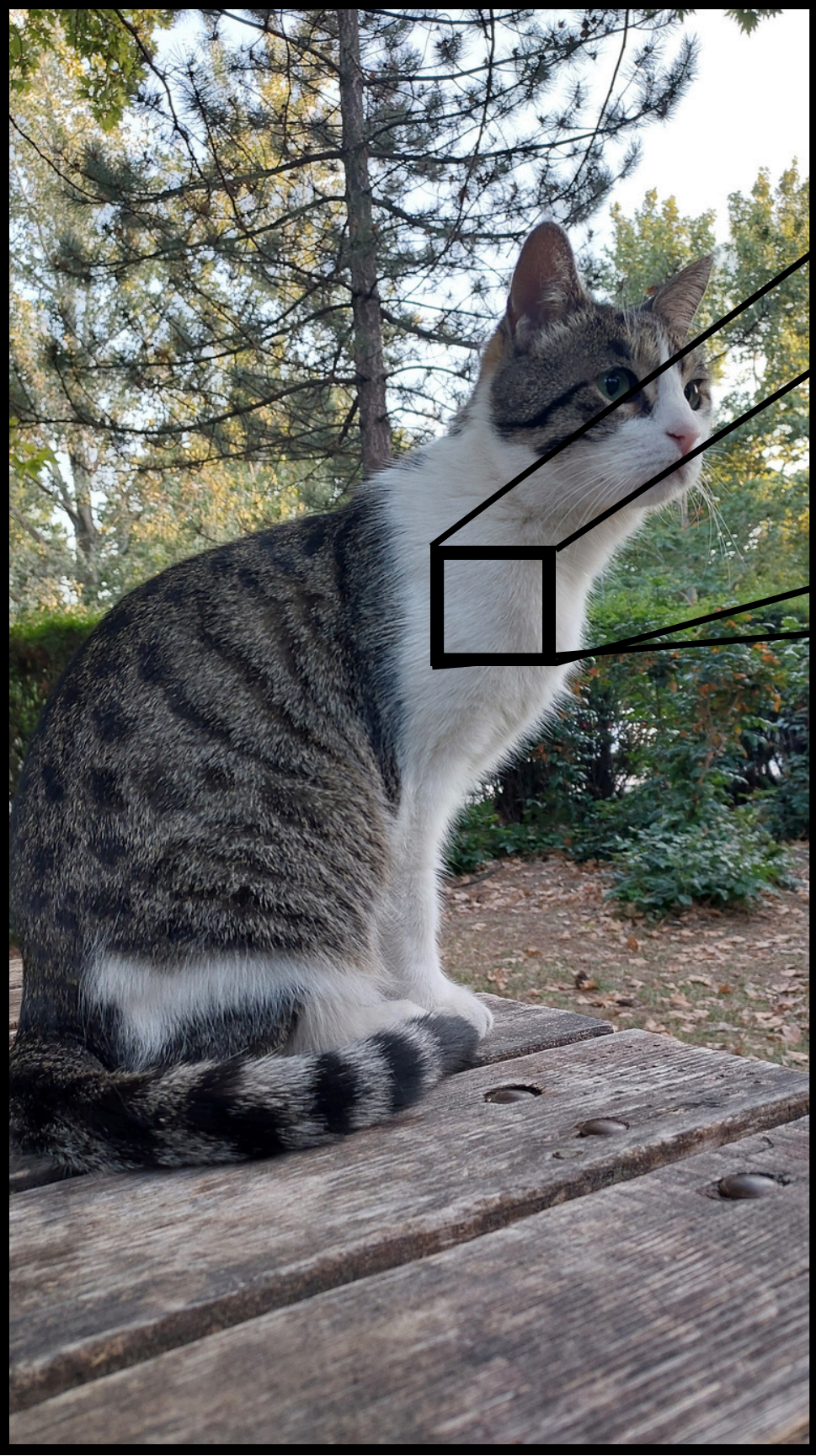
width = 2604 | height = 4624 | Channel/Depth = 3

height =4624



width = 2604

BGR



(4624, 2604,3)

width = 2604 | height = 4624 | Channel/Depth = 3

BGR	BGR	BGR
BGR	BGR	BGR
BGR	BGR	BGR
BGR	BGR	BGR

height =4624

$\begin{bmatrix} 160 & 147 & 145 \end{bmatrix}$	...	$\begin{bmatrix} 151 & 138 & 136 \end{bmatrix}$
$\begin{bmatrix} 140 & 128 & 126 \end{bmatrix}$	...	$\begin{bmatrix} 149 & 136 & 134 \end{bmatrix}$
	...	
$\begin{bmatrix} 144 & 131 & 129 \end{bmatrix}$	...	$\begin{bmatrix} 142 & 130 & 128 \end{bmatrix}$
$\begin{bmatrix} 144 & 131 & 129 \end{bmatrix}$	...	$\begin{bmatrix} 150 & 137 & 135 \end{bmatrix}$

width = 2604

Red Channel Gradient (0-255)



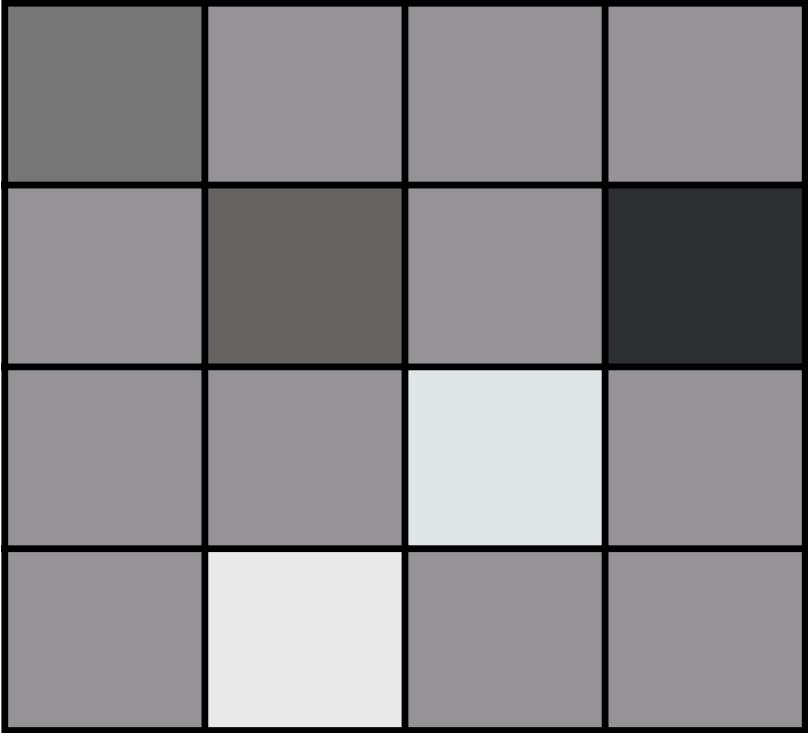
Green Channel Gradient (0-255)



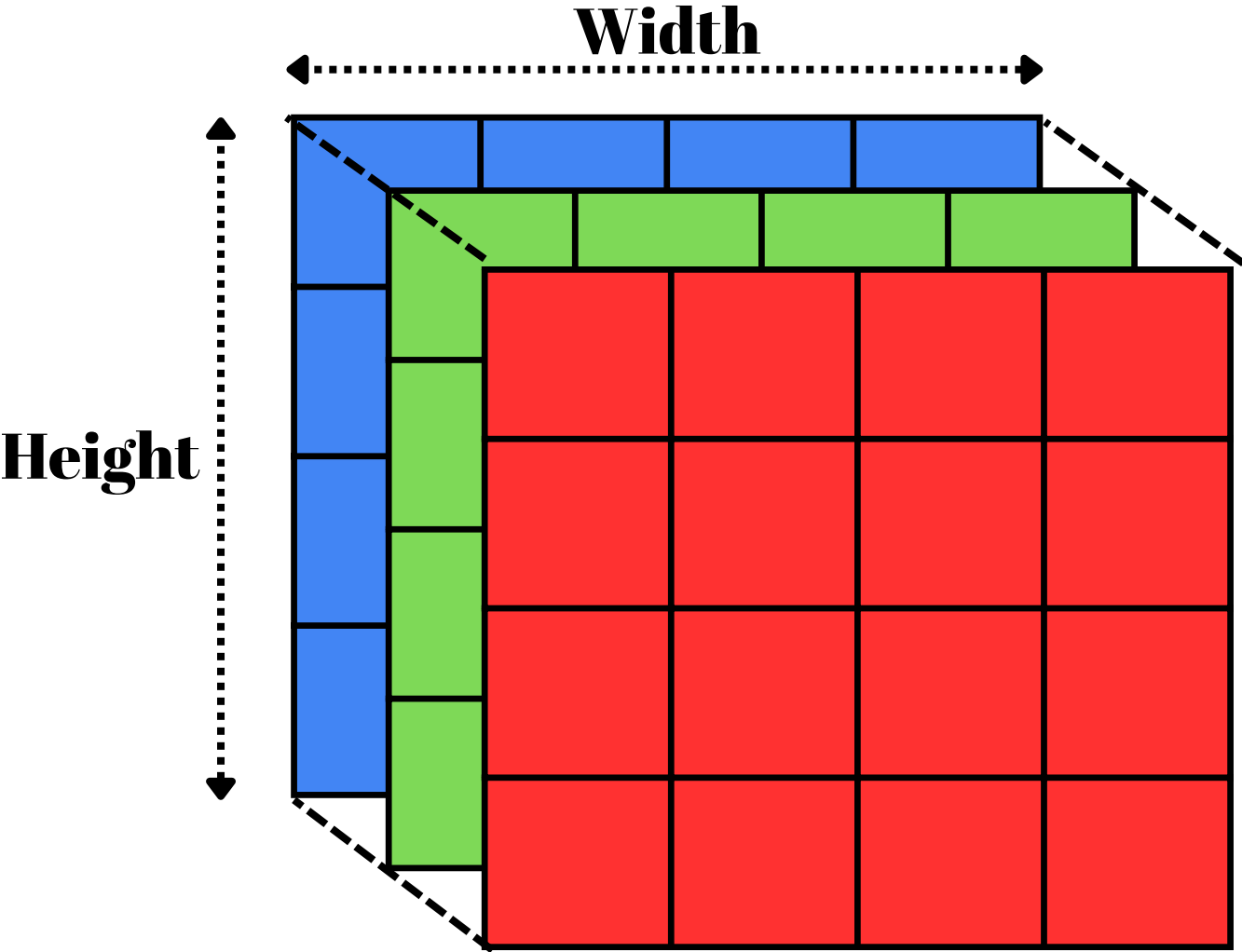
Blue Channel Gradient (0-255)



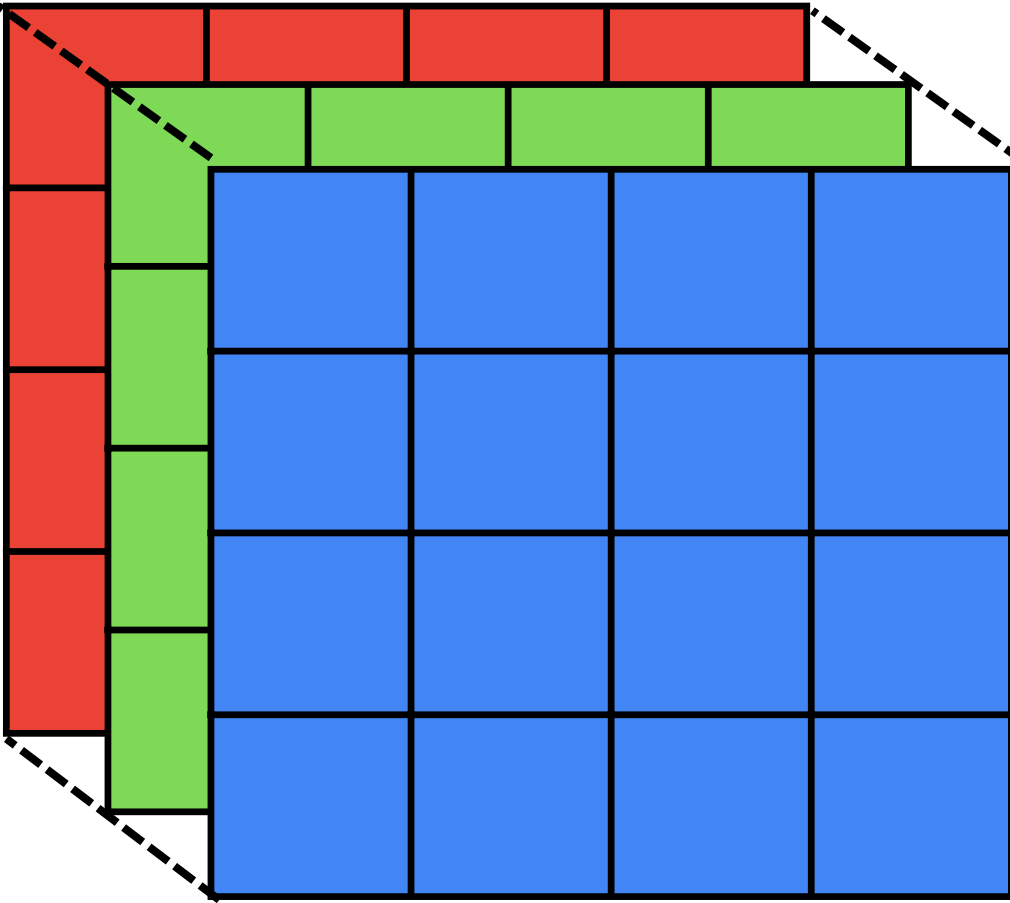
RGB-BGR



Gray Scale



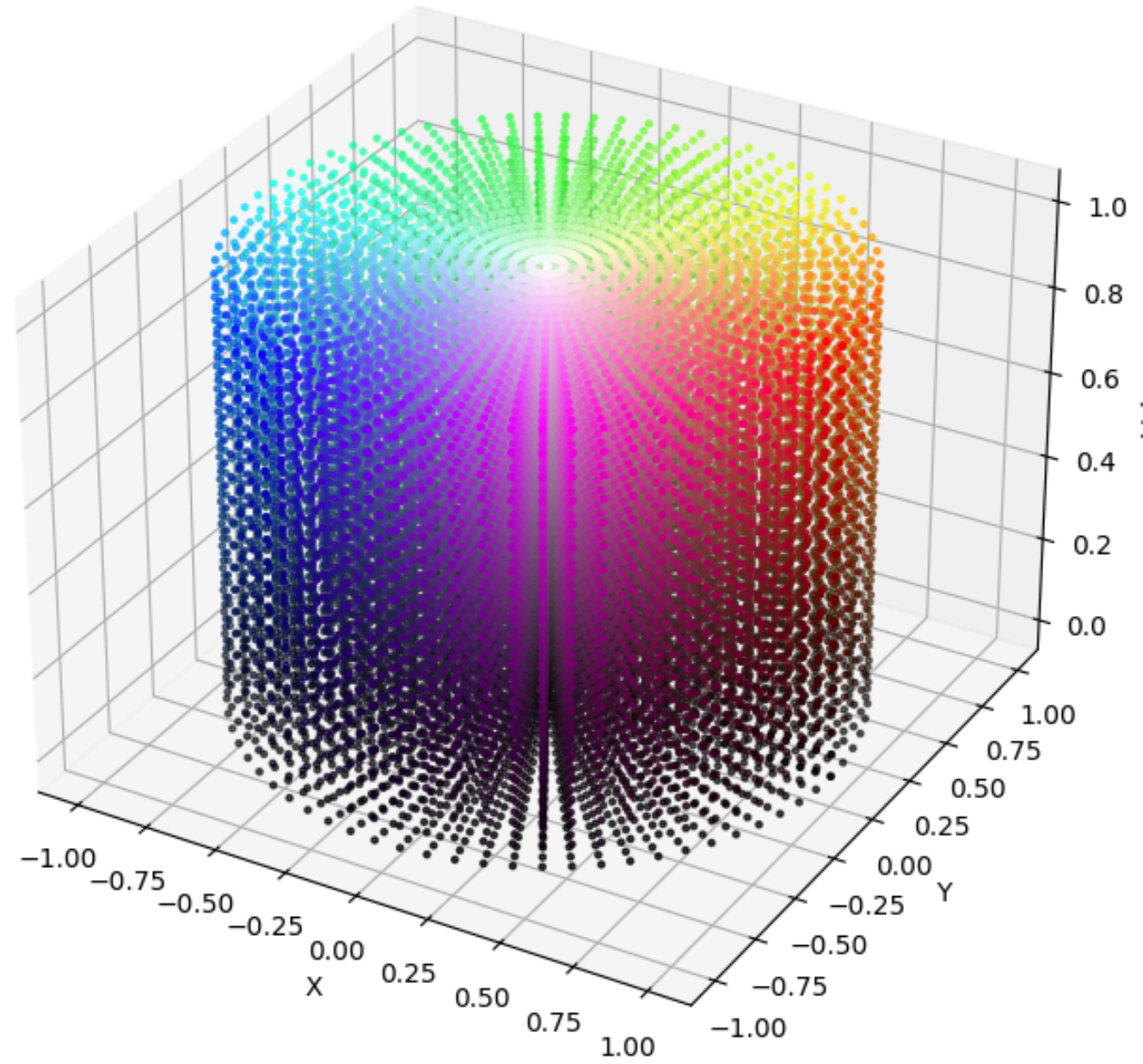
RGB



BGR

HSV-CMYK

HSV Color Space Cylinder

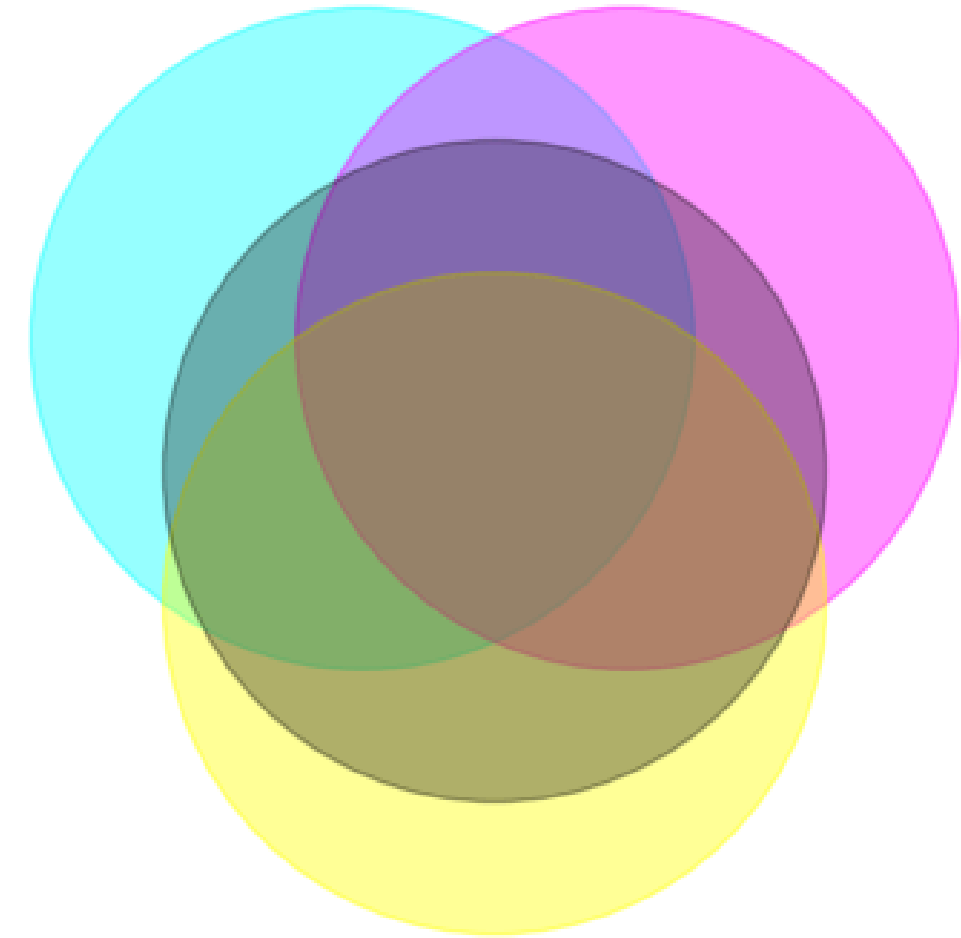


H (Hue) → renk tonu (0-360)

S (Saturation) → renk doygunluğu (0-1)

V (Value) → parlaklık / yoğunluk (0-1)

CMYK Ink Overlap (Venn-style Visualization)



C → Cyan

M → Magenta

Y → yellow

Black

BGR2Gray

$$\mathbf{G}_{ij} = \begin{bmatrix} \mathbf{0.114} & \mathbf{0.585} & \mathbf{0.301} \\ \mathbf{w_1} & \mathbf{w_2} & \mathbf{w_3} \end{bmatrix} \begin{bmatrix} \mathbf{R_{ij}} \\ \mathbf{G_{ij}} \\ \mathbf{B_{ij}} \end{bmatrix}$$

[111, 145, 145]	[109 143 143]	[67 102 105]
[130 164 164]	[127 161 161]	[97 132 135]
[147 181 180]	[142 176 175]	[123 159 159]

.....➔

141	139	99
160	157	129
177	172	155

Thresholding

Thresholding: Eşikleme işlemidir.

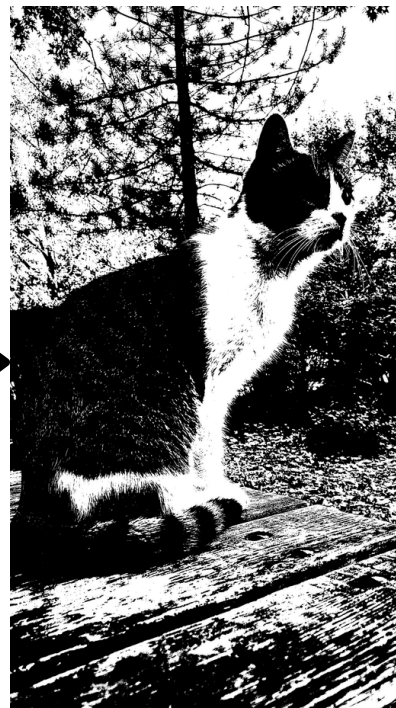
$$f(x) = \begin{cases} 255 & \text{if } x > T \\ 0 & \text{if } x \leq T \end{cases}$$

200	250	255
0	50	100
20	100	125



T = 125

255	255	255
0	0	0
0	0	0

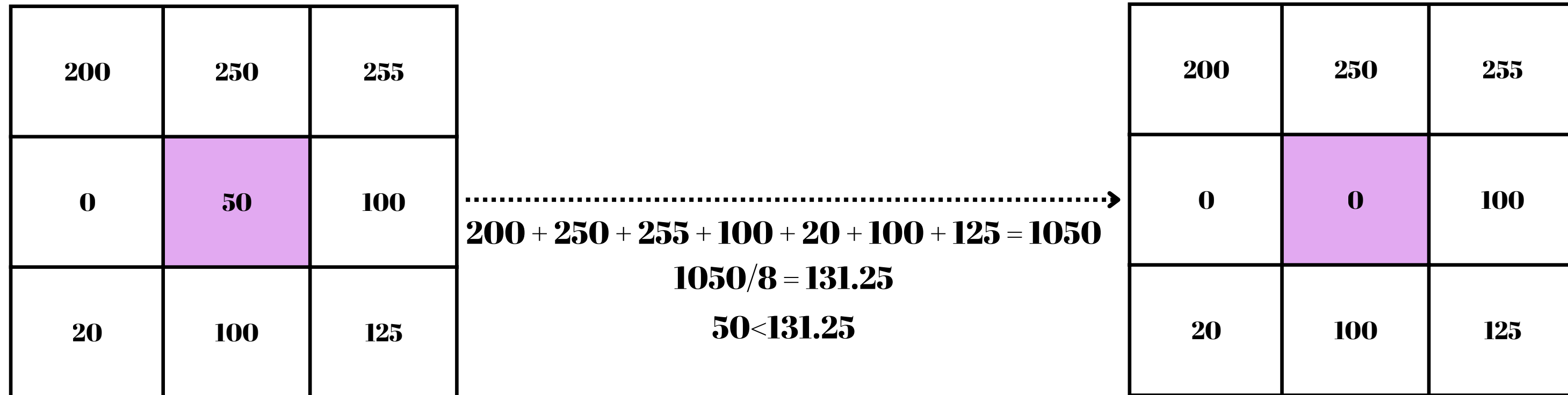


- Son durumda oluşan tensor sadece 0 ve 255 sayılarından oluşur.
- Arka plan-ön plan ayırma işlemlerinde kullanılır: Karakter tanıma, Barkod okuma, Belge tarama vb.
- RGB resimler için tek kanak bazlı veya 3 kanal birden uygulanabilir.

Adaptive Thresholding

$$f(x) = \begin{cases} 255 & \text{if } x > T \\ 0 & \text{if } x \leq T \end{cases} \longrightarrow T(x, y) = \frac{1}{N} \sum_{(i, j) \in \text{window}} I(i, j) - C$$

- C pikselleri daha fazla siyah veya daha fazla beyaz olarak etiketlemek için bir değer. Pozitif olması daha fazla beyaz etiketi, negatif olması daha fazla siyah etiketi anlamına gelir.**



- Adaptive thresholding lokal olarak thresholding yapar. Işıklandırmanın eşit dağılmadığı resimlerde daha avantajlıdır.**

Thresholding vs Adaptive Thresholding

Original



Global Threshold



Adaptive Mean



Adaptive Gaussian



- **Global Threshold lokal kontrastı yakalayamaz. Adaptive Mean ve Adaptive Gaussian bu konuda daha başarılıdır.**

Erosion

0	0	0	0	0
0	255	255	255	0
0	255	255	255	0
0	255	255	255	0
0	0	0	0	0

Tüm pixeller aynı değil → 0

0	0	0	0	0
0	255	255	255	0
0	255	255	255	0
0	255	255	255	0
0	0	0	0	0

Tüm pixeller aynı → 1

0	0	0	0	0
0	255	255	255	0
0	255	255	255	0
0	255	255	255	0
0	0	0	0	0

Tüm pixeller aynı değil → 0

$$E(x, y) = 1 \quad \text{if } \forall (i, j) \in B, I(x + i, y + j) = 1$$

$$E(x, y) = 0 \quad \text{otherwise}$$

0	0	0	0	0
0	0	0	0	0
0	0	255	0	0
0	0	0	0	0
0	0	0	0	0

Dilation

0	0	0	0	0
0	255	255	255	0
0	255	255	255	0
0	255	255	255	0
0	0	0	0	0

En az 1 adet 255 (1) var

0	0	0	0	0
0	255	255	255	0
0	255	255	255	0
0	255	255	255	0
0	0	0	0	0

En az 1 adet 255 (1) var

0	0	0	0	0
0	255	255	255	0
0	255	255	255	0
0	255	255	255	0
0	0	0	0	0

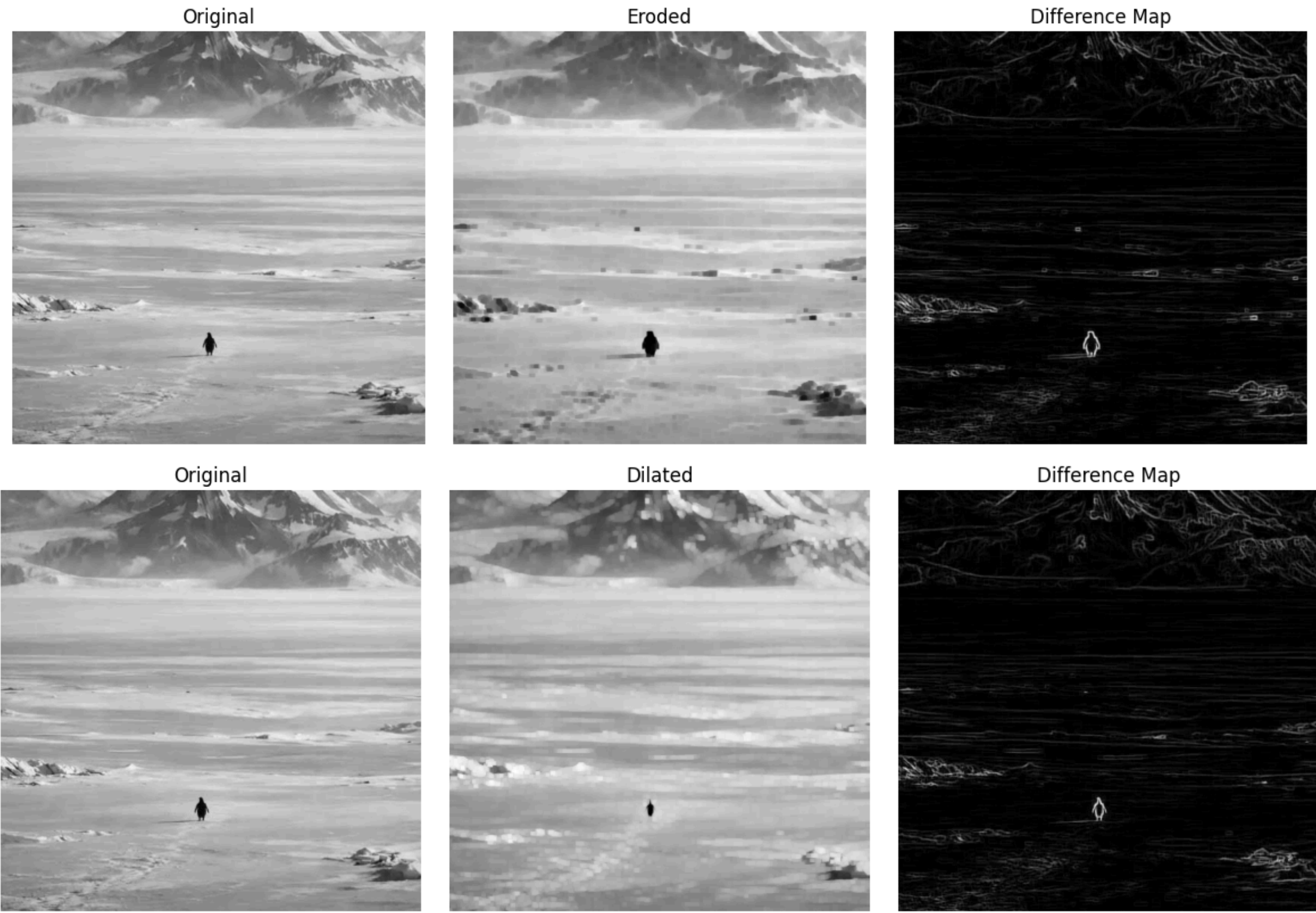
En az 1 adet 255 (1) var

$$D(x, y) = 1 \quad \text{if } \exists(i, j) \in B, I(x + i, y + j) = 1$$

$$D(x, y) = 0 \quad \text{otherwise}$$

255	255	255	255	255
255	255	255	255	255
255	255	255	255	255
255	255	255	255	255
255	255	255	255	255

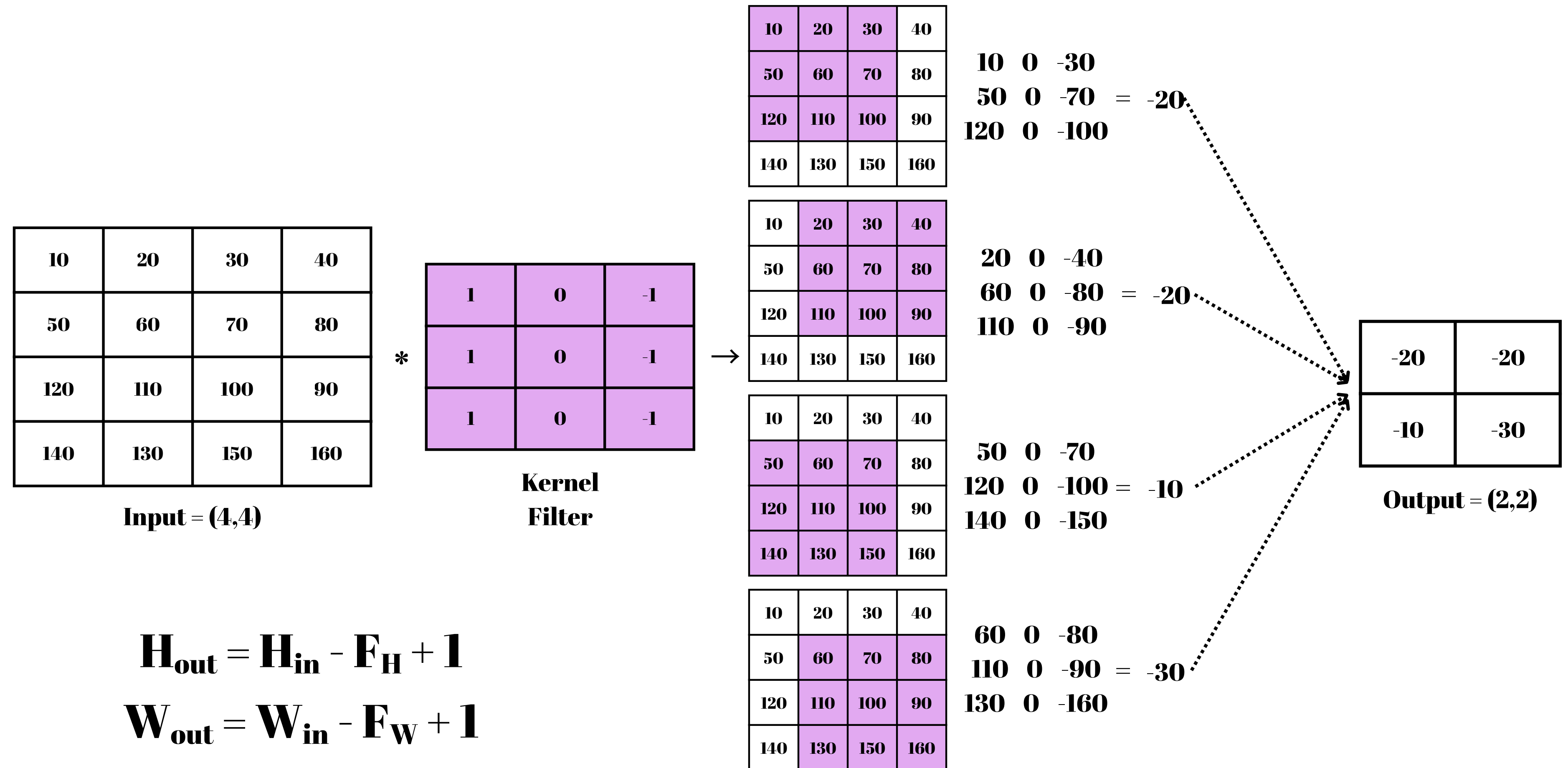
Erosion vs Dilation



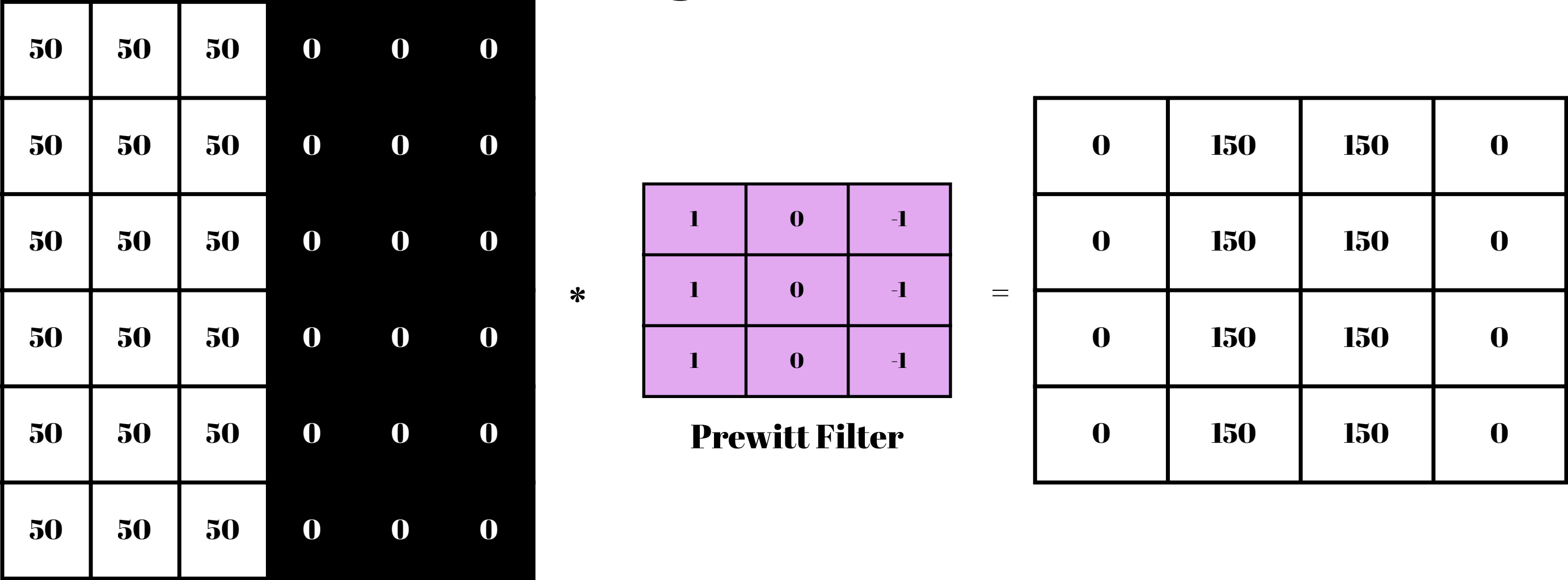
Erosion	Dilation
Nesneleri (Beyazları) küçültür (shape shrinking)	Nesneleri (Beyazları) büyütür
Küçük gürültüleri temizler (noise removal)	Küçük boşlukları doldurur.
Bozuk parçaları koparır	Bozuk parçaları birleştirir
Boundary (sınır) çıkarımında kullanılır.	Boundary + gap filling işlemleri

- Erosion ve dilation genelde “mask” yapısı üzerinde kullanılırlar.

Convolution Operation



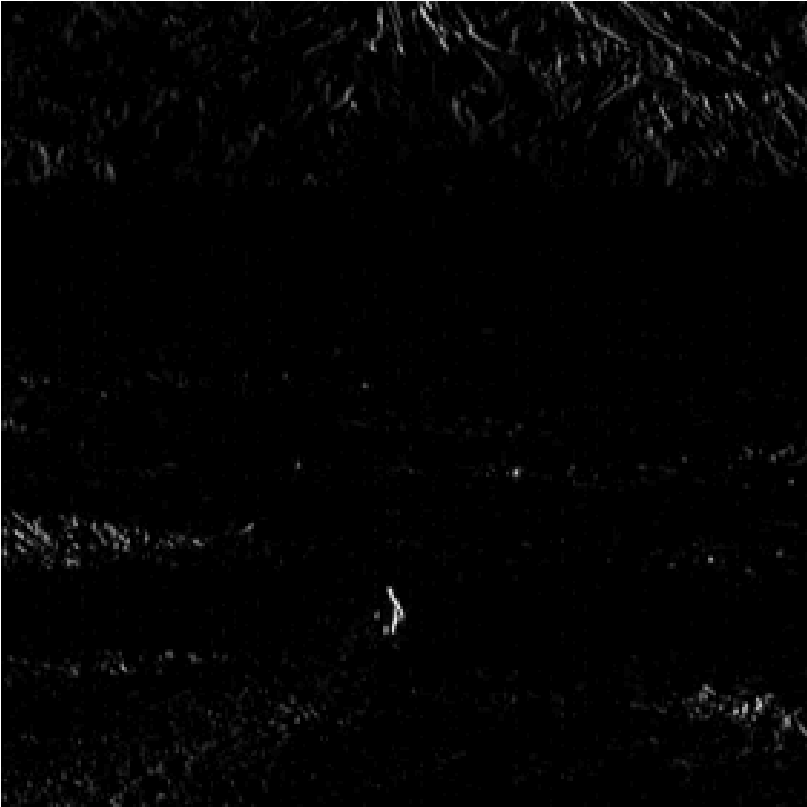
Edge Detection



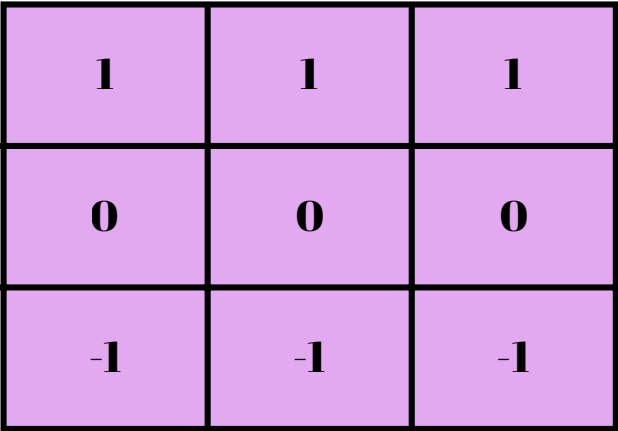
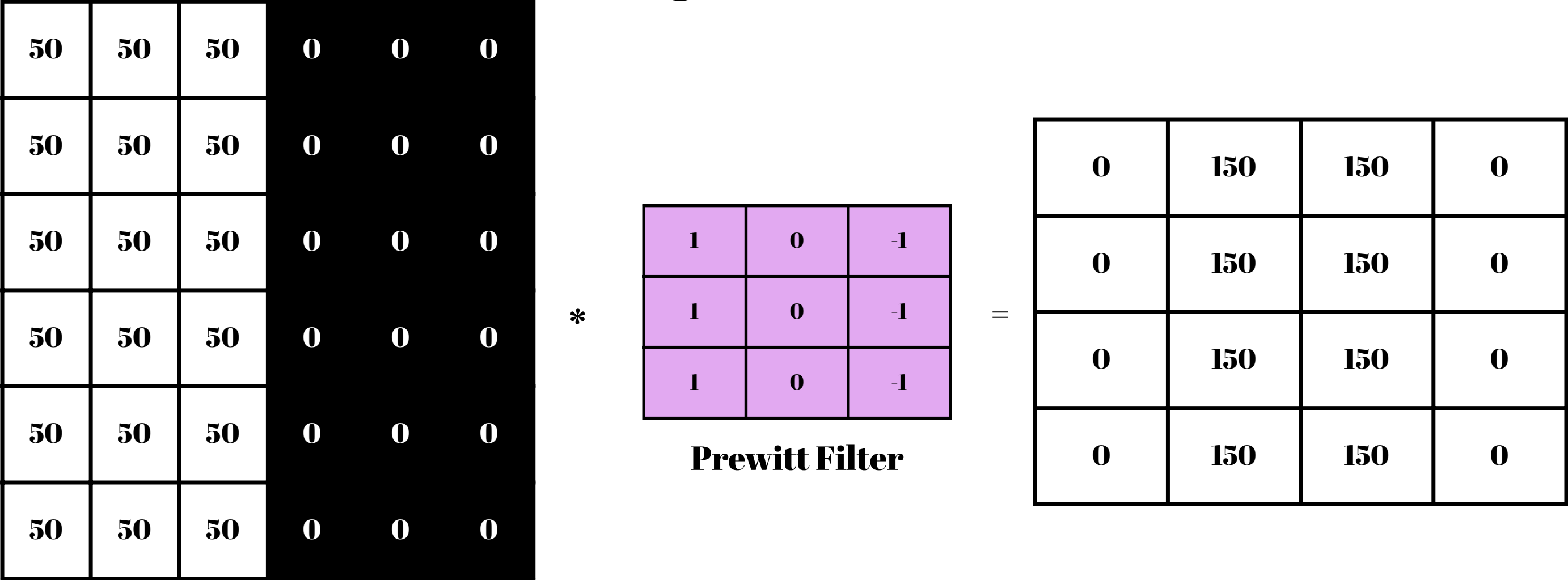
Original

Prewitt X

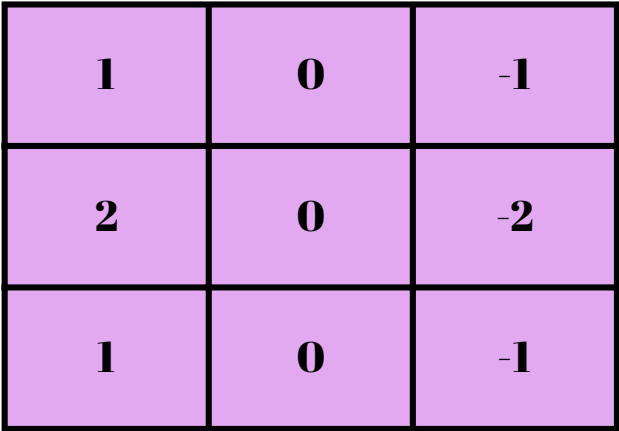
Prewitt Y



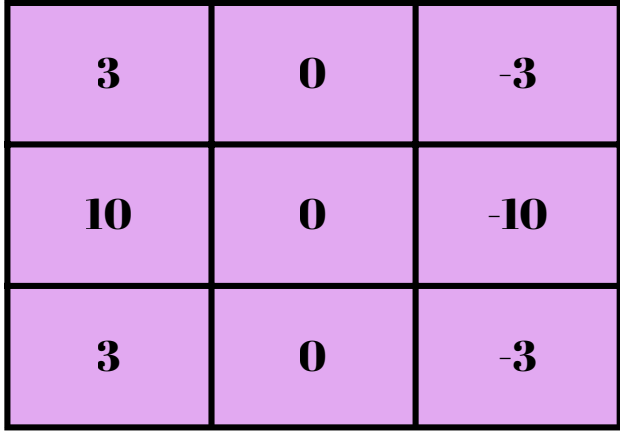
Edge Detection



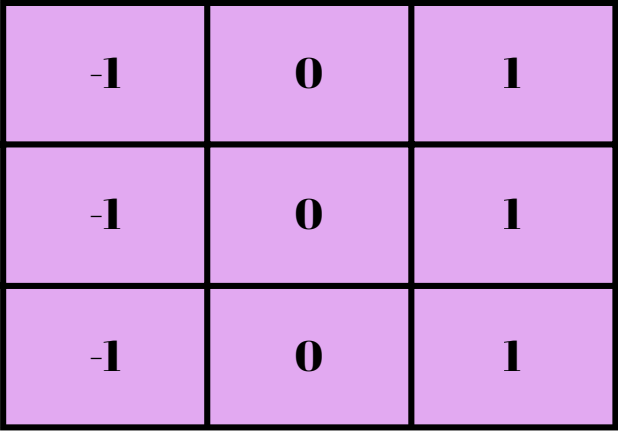
Vertical Edge



Sobel Filter

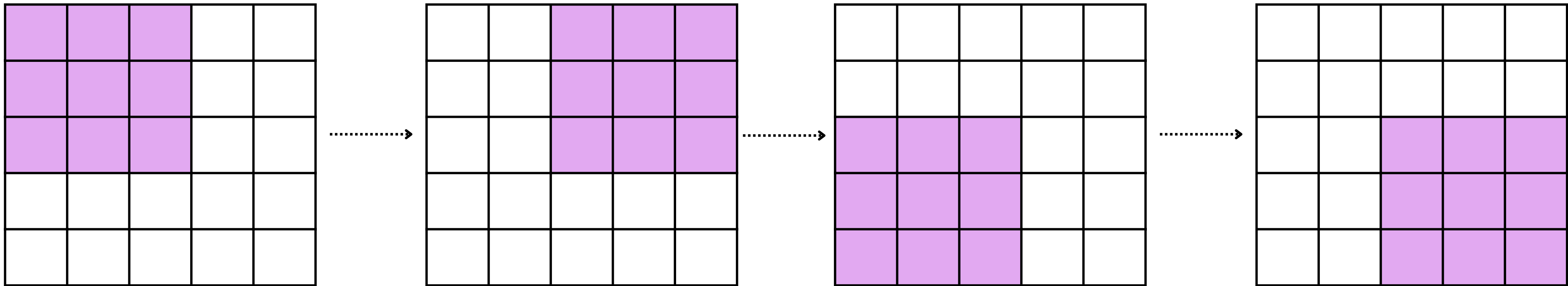


Scahrr Filter



Negative Filter

Stride

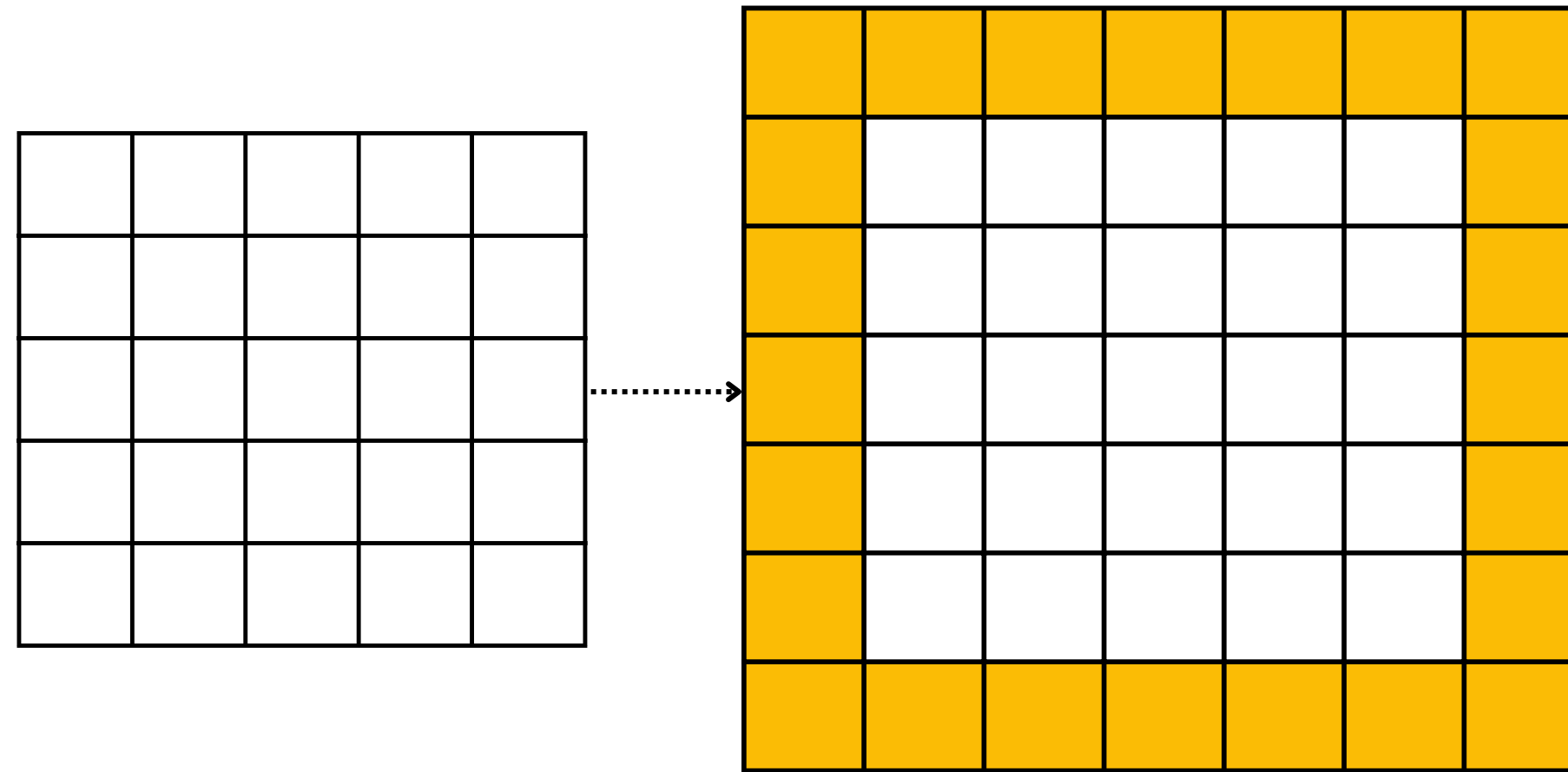


stride (s) = 2

$$\left[\mathbf{H}_{\text{out}} = \frac{\mathbf{H}_{\text{in}} - \mathbf{F}_{\text{H}}}{s} + \mathbf{1} \right]$$

$$\left[\mathbf{W}_{\text{out}} = \frac{\mathbf{W}_{\text{in}} - \mathbf{F}_{\text{W}}}{s} + \mathbf{1} \right]$$

Padding



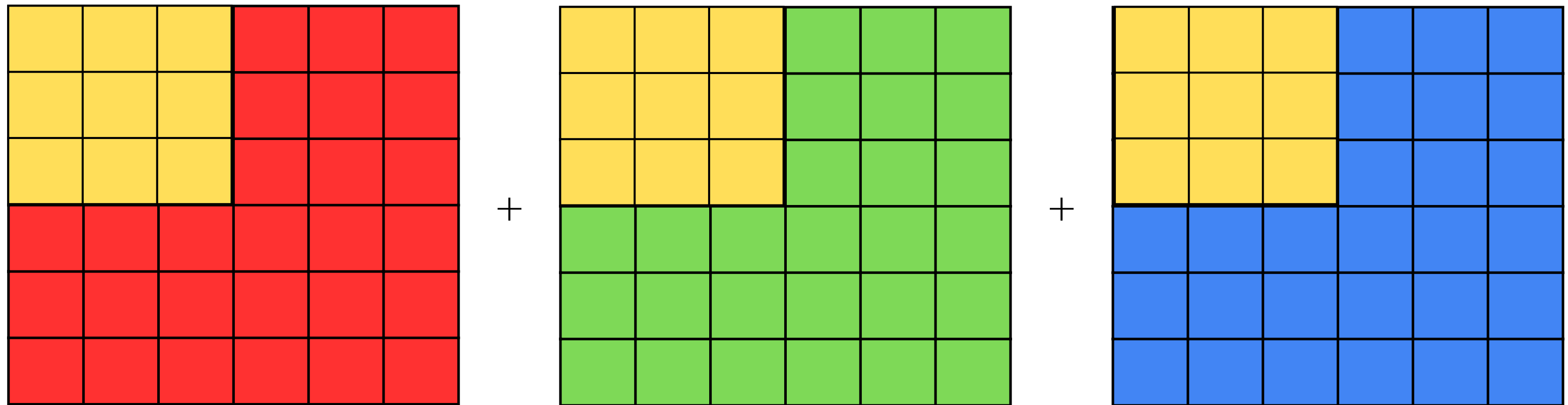
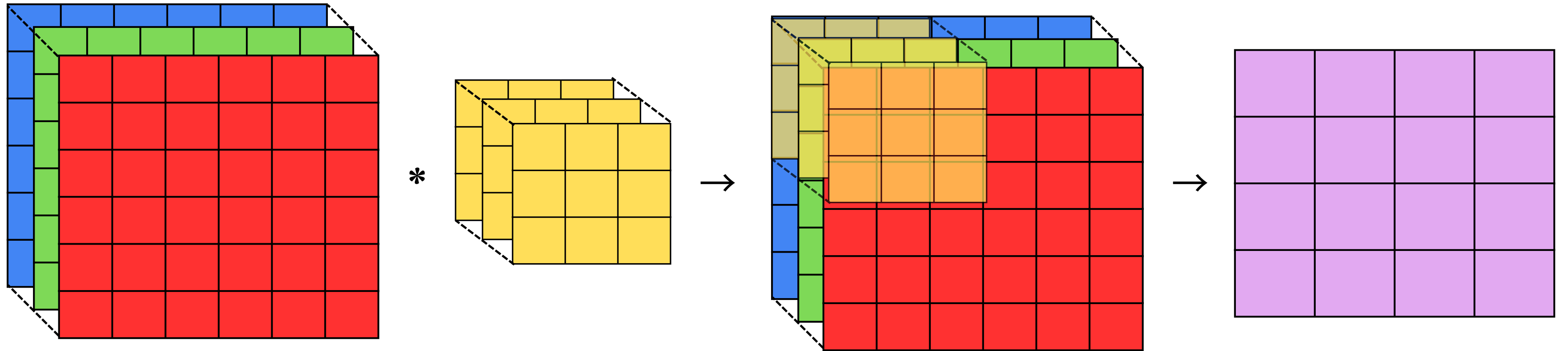
- **Shrinking the output: Output her convolution işleminde küçülüyor.**
- **Her piksel output'a eşit derecede katkı sağlamıyor.**
- **Genelde padding pixellerine 0 değeri atanır.**

$$\left\lceil \mathbf{H}_{\text{out}} = \frac{\mathbf{H}_{\text{in}} + 2\mathbf{P}_h - \mathbf{F}_H}{\mathbf{S}} + 1 \right\rceil$$

$$\left\lceil \mathbf{W}_{\text{out}} = \frac{\mathbf{W}_{\text{in}} + 2\mathbf{P}_h - \mathbf{F}_h}{\mathbf{S}} + 1 \right\rceil$$

- **valid:** Padding yoktur.
- **same:** orijinal input boyutu korunur.
- **full:** kernel input'un dışına kadar kayar. Output inputtan büyüktür

3D Convolution



3D Convolution

10	0	0	...	...	...
10	0	0	...	...	...
10	0	0	...	...	...
...	...	...	...	...	...
...	...	...	...	...	...
...	...	...	...	...	...

*

1	0	-1
1	0	-1
1	0	-1



30	...	...	...
...	...	...	...
...	...	...	...
...	...	...	...

0	0	20	...	...	...
0	0	20	...	...	...
0	0	20	...	...	...
...	...	...	...	...	...
...	...	...	...	...	...
...	...	...	...	...	...

*

1	0	-1
1	0	-1
1	0	-1



-60	...	...	...
...	...	...	...
...	...	...	...
...	...	...	...

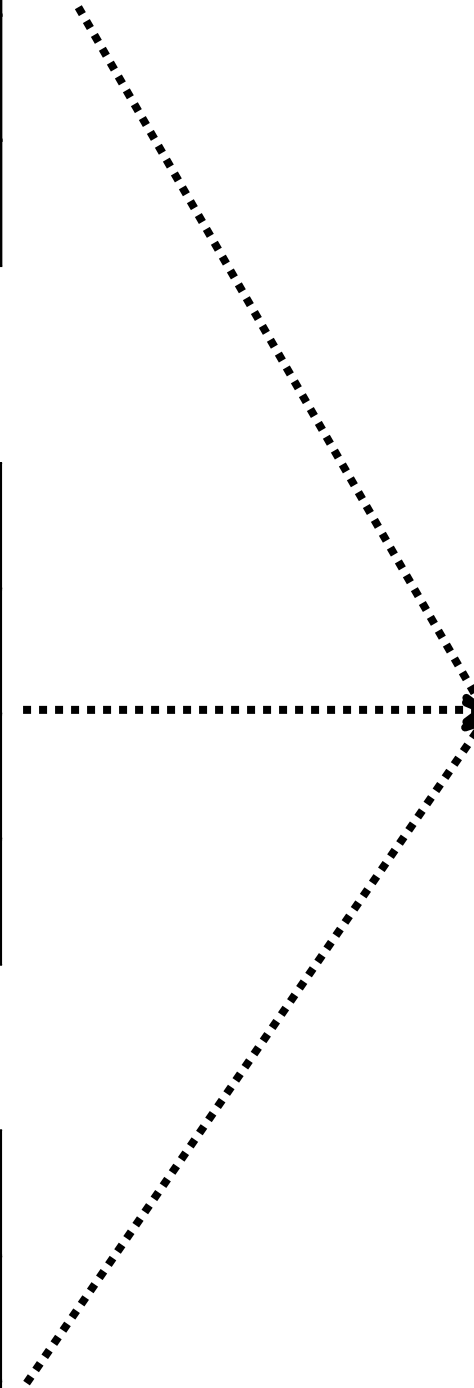
0	0	0	...	...	...
0	0	0	...	...	...
0	0	0	...	...	...
...	...	...	...	...	...
...	...	...	...	...	...
...	...	...	...	...	...

*

1	0	-1
1	0	-1
1	0	-1



0	...	...	...
...	...	...	...
...	...	...	...
...	...	...	...



-30	...	...	...
...	...	...	...
...	...	...	...
...	...	...	...

Blurring/Smoothing

Mean Blurring

0	0	0	0	0	0
0	10	20	30	40	0
0	50	60	70	80	0
0	120	110	100	90	0
0	140	130	150	160	0
0	0	0	0	0	0

padding = same

$\ast 1/9$

1	1	1
1	1	1
1	1	1

15.5	26.6	...	...
...	...	...	...
...	...	...	...
...	...	...	...

- Gürültüleri ve ani pixel değişimlerini yumuşatır
- Overfitting'i önleyebilir.
- Bilgi kaybı riski vardır.

Median Blurring

0	0	0	0	0	0
0	10	20	30	40	0
0	50	60	70	80	0
0	120	110	100	90	0
0	140	130	150	160	0
0	0	0	0	0	0

padding = same

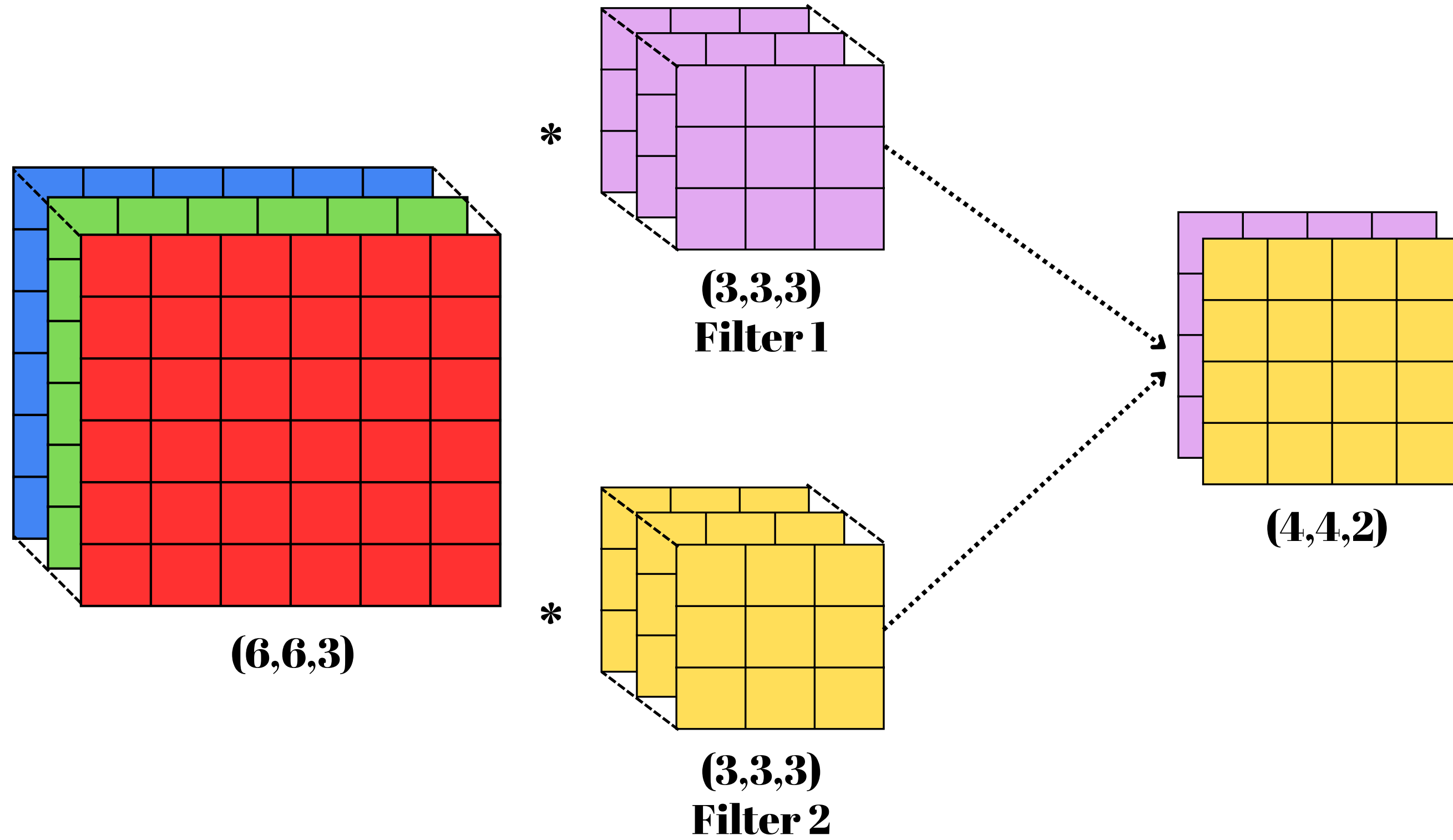
$\ast$

Medianı bul

.....>

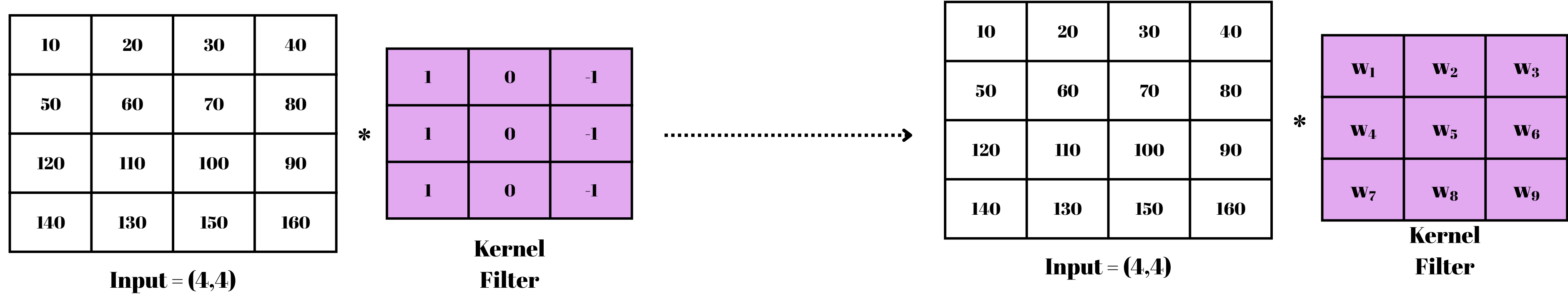
0	20	...	...
...	...	...	...
...	...	...	...
...	...	...	...

Multi-Filter Convolution



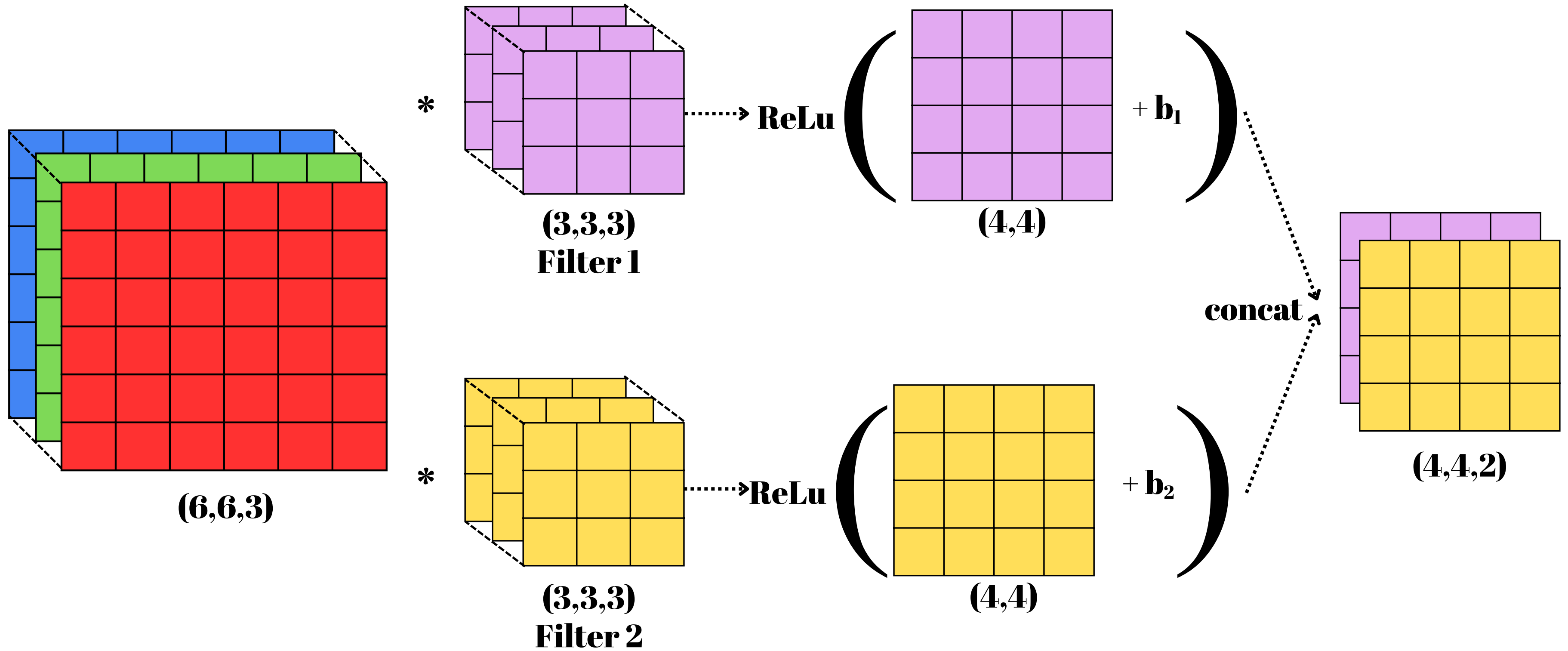
- Birden fazla filtre uygulanması farklı özelliklerin çıkartılmasını sağlayabilir. Örnek olarak yatay ve dikey kenar tespiti vb.

Learning Parameters

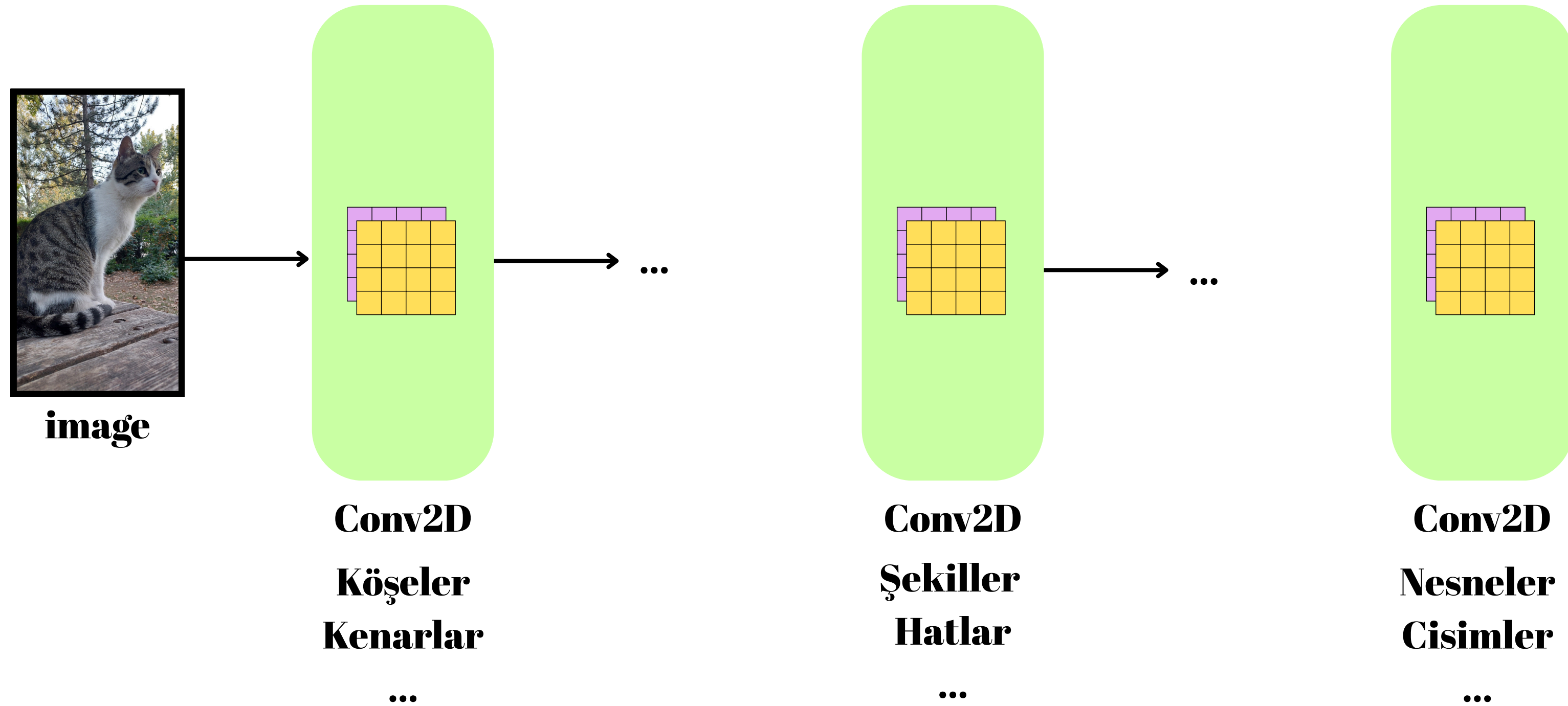


- **Conv filter üzerindeki sayılar NN ve backpropagation yardımı ile öğrenilebilir.**
- **Model eğitim süreci boyunca kenarları, köşeleri, mantıklı şekilleri vb. ortaya çıkaracak filtre parametrelerini öğrenebilir. (Feature Extraction)**

Convolution Layer



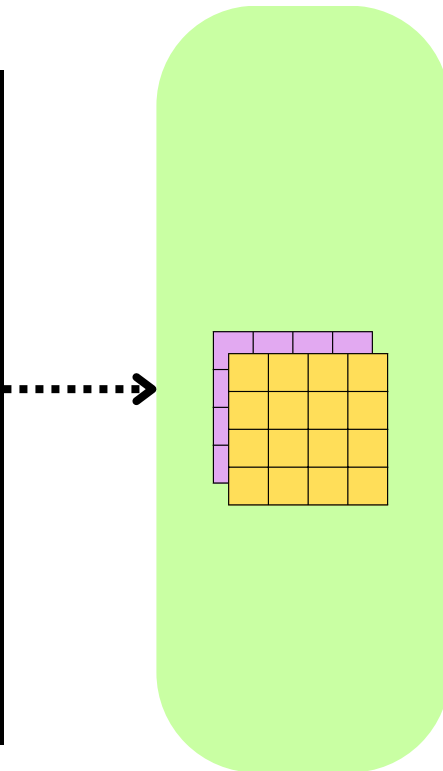
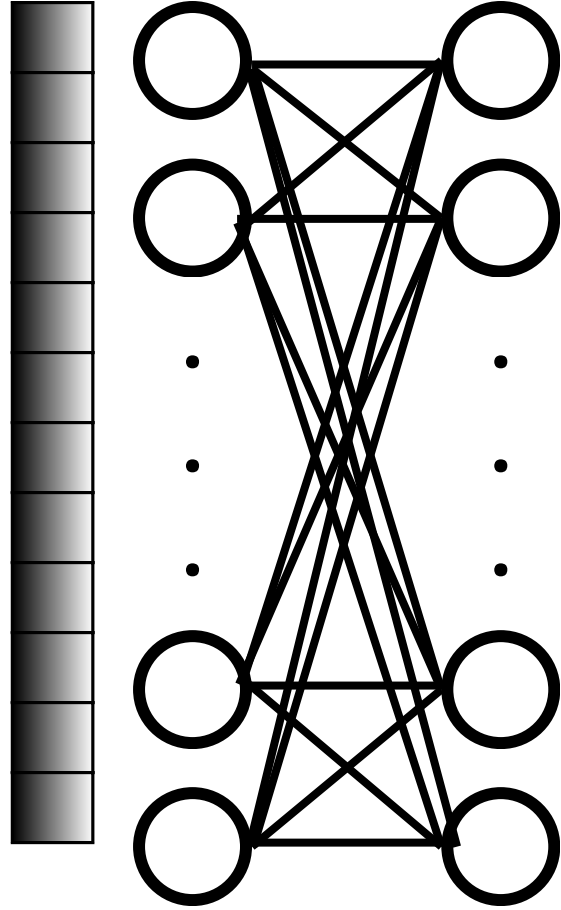
Learning Objects



Motive for CNN

- Sabit bir girdi boyutu
- Pixeller arasında ilişki kurulmaz. Çevre pixellere bakılmaz

NN	CNN
Dense Connectivity: Her nöron kendinden önceki katmandaki tüm nöronlara bağlıdır.	local receptive field: Her pixel çevre pixel'ler ile beraber filtre içerisinde değerlendirilir.
Weight Sharing yoktur. Çok fazla parametre vardır	Weight Sharing vardır. Parametre sayısı görece daha azdır.
Flatten uygulanır. Uzaysal (spatial) bilgi yok olur.	Spatial bilgi korunur.
Nesnenin pozisyonundaki değişimler NN performansını ciddi derecede etkileyebilir.	Nesne pozisyonundaki değişimler ciddi sorun yaratmaz çünkü kernel bütün resmi dolaşır.



Pooling

Pooling: Özellikleri korurken boyutu küçültür.

Max Pooling:
window = 2

3	8	1	7
5	2	6	3
5	4	2	7
10	8	6	7

.....→

8	7
10	7

Average Pooling:
window = 2

3	8	1	7
5	2	6	3
5	4	2	7
10	8	6	7

.....→

18/4	17/4
27/4	22/4

- **Network içindeki parametre sayısını azaltır.**
- **Feature extraction daha sağlam olur.**
- **Overfitting önleyebilir.**
- **Parametre (weight,bias) yoktur. Backpropagation sırasında öğrenilmez.**
- **Max Pooling tespit edilen özelliği (max) korur diğerlerini siler.**

Pooling

Sum Pooling:
window = 2

3	8	1	7
5	2	6	3
5	4	2	7
10	8	6	7



18	17
27	23

Stochastic Pooling
window = 2

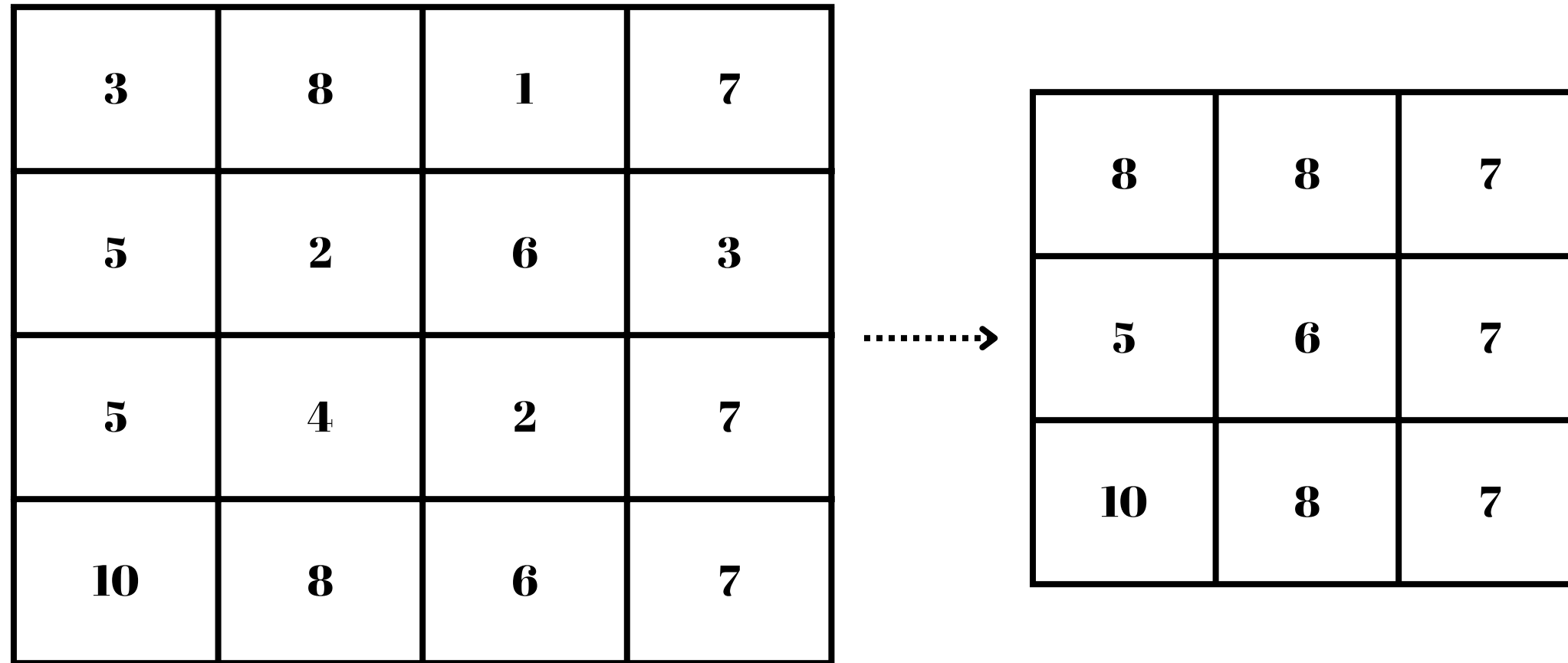
3	8	1	7
5	2	6	3
5	4	2	7
10	8	6	7



2	7
10	6

- **Pool içindeki sayılar toplanır.**
- **Random bir pixel'in değeri seçilir.**
- **Seçim yaparken farklı olasılıksal modeller kullanılabilir.**

Overlapping Pooling

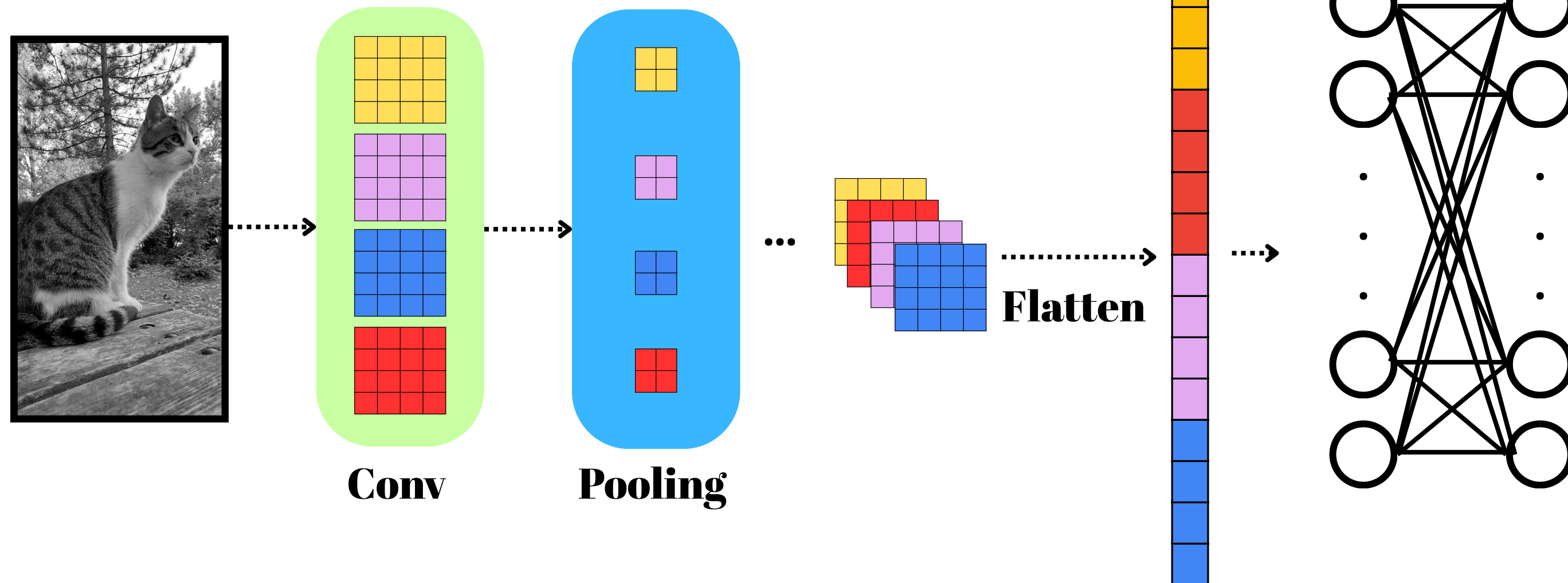


Overlapping Max Pooling:

window = 2

- **Overlapping pooling olması için stride < size olmalıdır ($s < z$)**

Fully Connected Layer



Last Layer

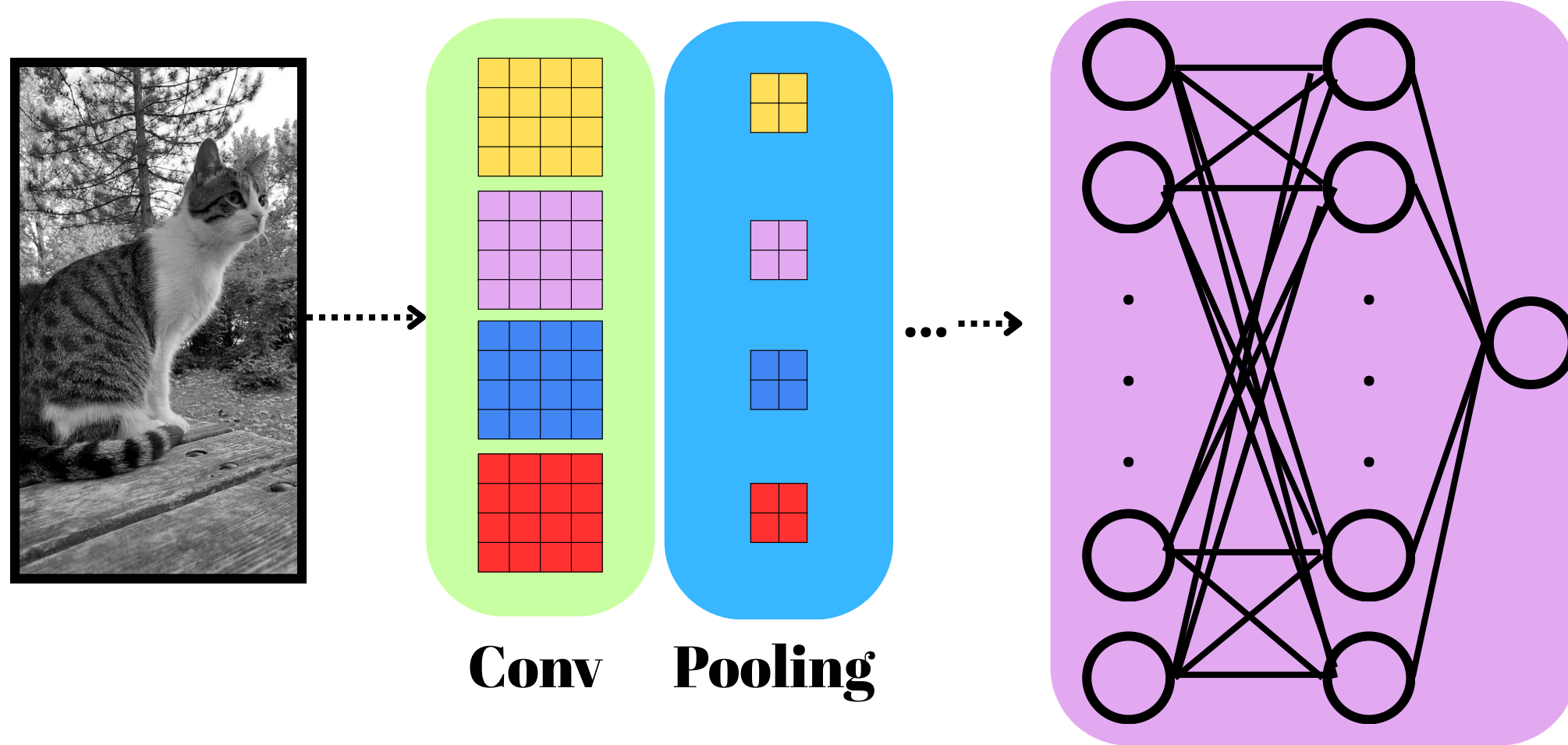
☐ **Binary Classification**

☐ **Multiclass
classification**

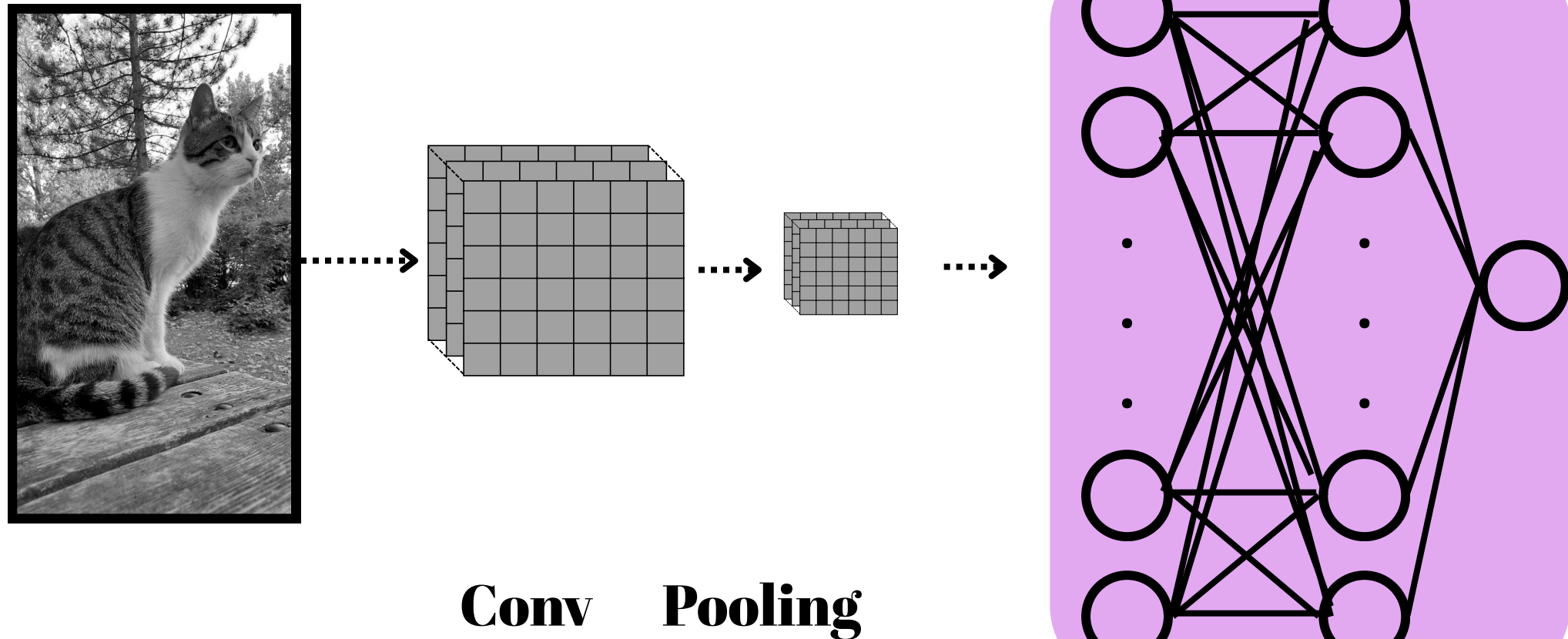
☐ **Regression**

☐ **Multi Regression**

Convolutional Neural Network



- **Conv Layer özellikleri yakalar. Pooling layer boyut azaltır. FC Layer çıkartılan özelliklerden tahmin yapar.**

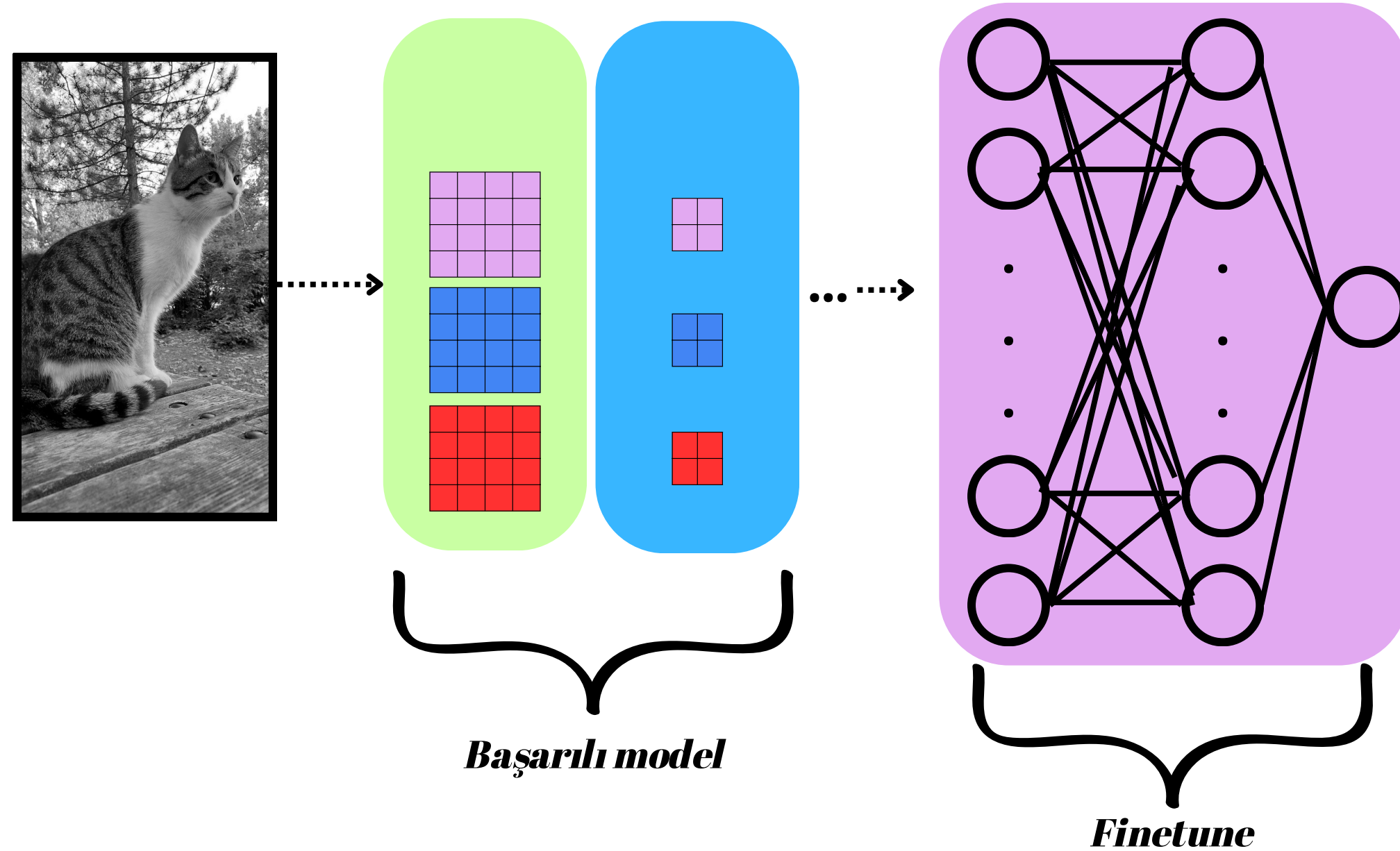


Convolutional Neural Network

Shallow CNN	Deep CNN
Çok fazla Conv Block içermez. Parametre sayısı azdır.	Fazla Conv Block içerir. Parametre sayısı fazladır.
Underfitting riski vardır	Daha fazla öğrenme kapasitesi vardır. Overfitting riski vardır. Vanishing/Exploding Gradient problemi yaşanabilir. *Derin CNN mimarilerinde overfitting problemini çözebilecek farklı uygulamalar vardır (Batch norm vb.) Derin modellerin performans düşüşü doğrudan overfitting ile ilgili değil daha sonra bahsedilecek “degradation problem” ile ilgili olabilir.
Eğitim süresi kısadır.	Eğitim süresi uzundur

Transfer Learning

- **Amaç:** Sıfırdan model eğitmek yerine önceden eğitilmiş modelleri kullanmak.



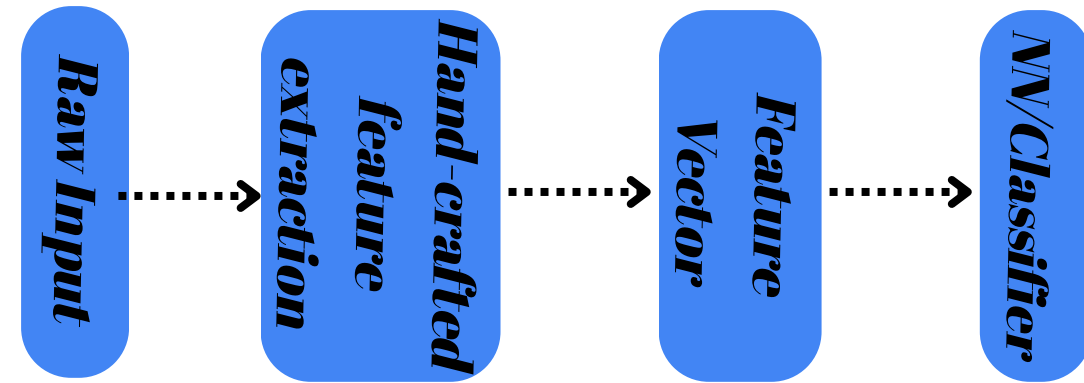
- **Pretrained model:** Daha önceden büyük veriler üzerinde eğitilmiş ve başarı göstermiş modellerdir.
- **Finetune:** Pretrained modellerin sadece son katman(lar)ını eğiterek istediğimiz probleme özel duruma getirmektir.
- Eğitilmeyen katmanlar dondurulur (**freeze**) ve bu katmanlar üzerinde gradyan hesaplama/parametre güncelleme işlemleri gerçekleştirilmez.

- Küçük veri seti içeren durumlarda faydalıdır.
- Yüksek performanslı başlangıç sağlar.
- Finetune sırasında yüksek learning rate sıkıntı yaratabilir.

LeNet-5

*YANN LECUN, MEMBER, IEEE, LEON BOTTOU, YOSHUA BENGIO, ' AND PATRICK HAFFNER. 1998. Gradient-Based Learning Applied to Document Recognition

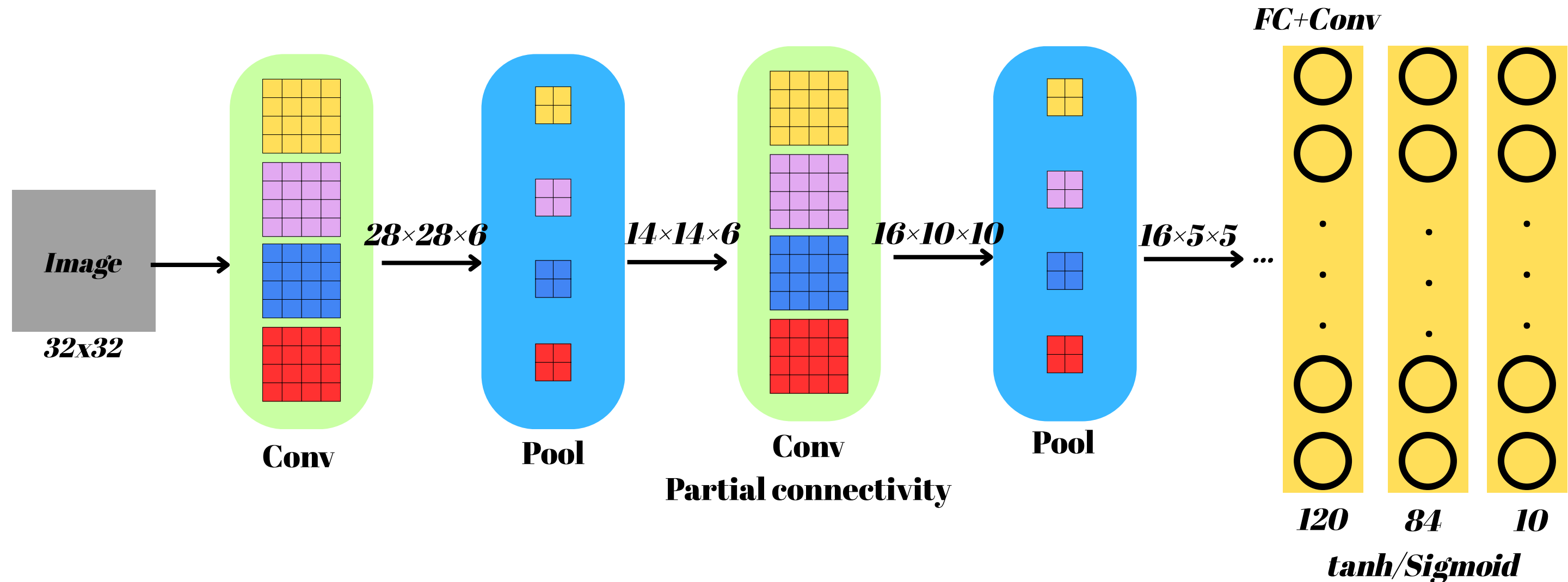
- Bu paper'a kadar olan görüntü işleme pipeline'ları



Otomatik öğrenilemez, Developer'ın el ile özellikleri çıkartması lazım.
End-to-End değildir. Algoritma tek adımda çalışmaz.

- LeNet bugünkü Conv ve Pooling'in temellerini atmıştır.

Dataset: MNIST



AlexNet

**Alex Krizhevsky, Ilya Sutskever, Geoffrey E. Hinton. 2012. ImageNet Classification with Deep Convolutional Neural Networks*

- ImageNet yarışması için eğitilmiş bir model. Önerildiği yıla kadar olan bütün modellerden daha iyi performans gösterdi.

Data

- 1.2 yüksek çözünürlüklü resim, 1000 class output

Preprocess:

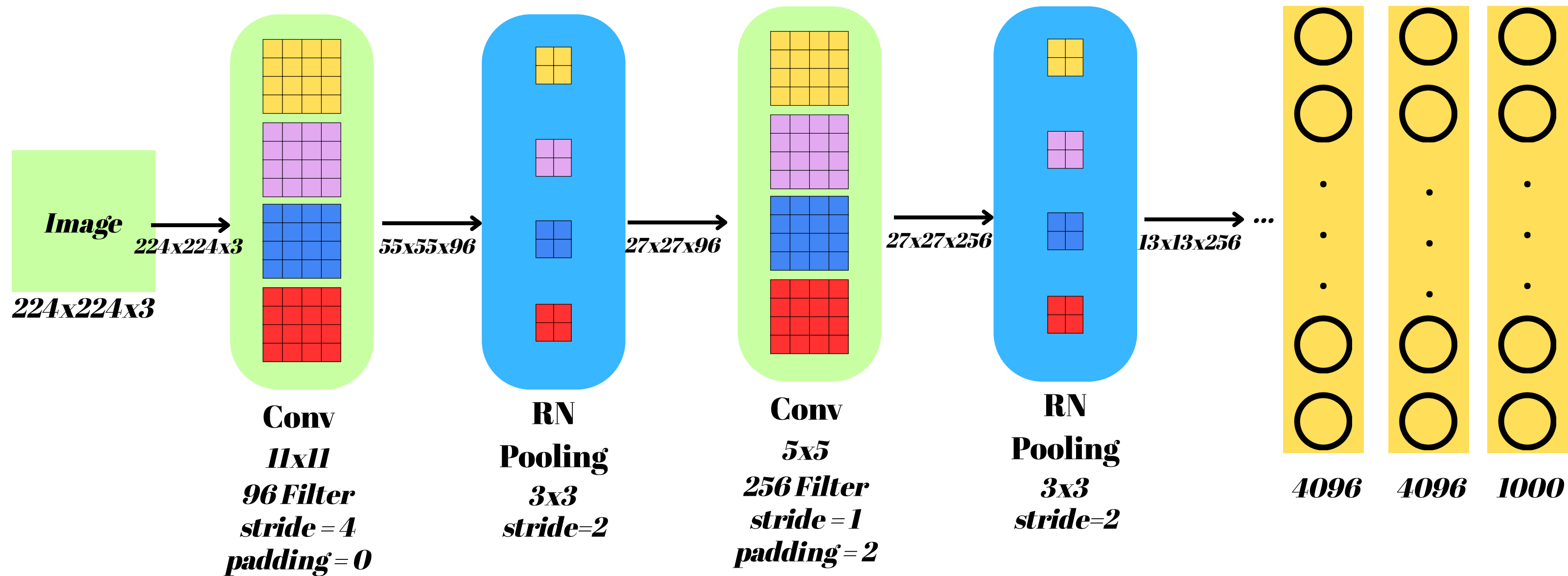
- *Kısa kenar 256 olacak şekilde rescale yapılmış. daha sonra 256x256 olacak şekilde resim kırpılmış.*
- *Train-time içerisinde “Translation, Flip” ve “RGB insensity değişimi” gibi Data augmentation yöntemleri uygulanmıştır.*

Model

- Modelin önemli yeniliklerinden biri aktivasyon fonksiyonu olarak **ReLU** kullanmak. Bu modele kadar ağırlıklı olarak sigmoid ve tanh kullanılıyordu ancak bu modelde eğitimi daha hızlı olduğu için ReLU tercih edilmiş. (açık bir şekilde belirtilmese de performansa da katkı sağladığını söyleyebiliriz.)
- Kernellerin üst yarısı 1. GPU'ya, alt yarısı 2. GPU'ya konularak eğitimin paralelleştirilmesi sağlanmış.
- ReLU çıktılarına channel-wise normalization (**Local Response Normalization**) uygulanmıştır. Bu normalization sadece bir filtre üzerinde değil bütün filtreler baz alınarak uygulanır. Özelliklerin daha sağlam çıkartılmasını sağlar.
- Overlapping (s=2, z=3) max pooling kullanılmıştır.
- İlk iki FC Layer için dropout kullanılmış.

²AlexNet

**Alex Krizhevsky, Ilya Sutskever, Geoffrey E. Hinton. 2012. ImageNet Classification with Deep Convolutional Neural Networks*



VGG

****Karen Simonyan, Andrew Zisserman.2014.Very Deep Convolutional Networks for Large-Scale Image Recognition***

- ImageNet yarışmasında 1.
- Temel arayış CNN derinliği arttıkça accuracy artar mı?

Preprocess:

- **resize**
- **random crop 224x224**
- **horizontal flip**
- **RGB mean subtraction**

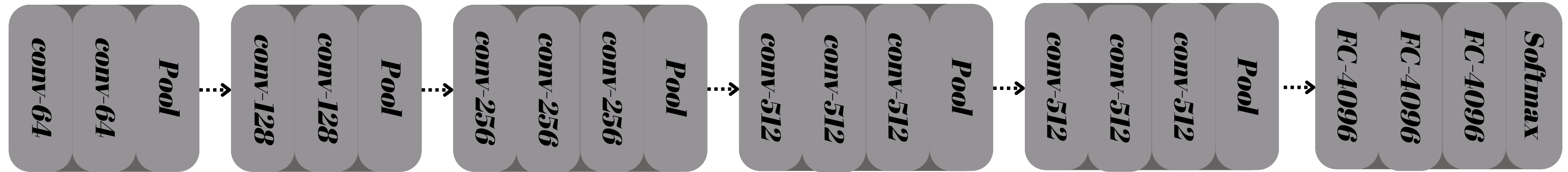
Model

- **Homojen bir model kurgulanmıştır.**
 - **3x3 convolution**
 - **stride = 1**
 - **padding = same**
 - **pooling = 2x2 maxpool stride 2**
- **Derinlikleri farklı modeller kurgulanmıştır. (A->E) (11->19)**
- **3x3 conv kullanıldığı için parametre sayısı azdır.**

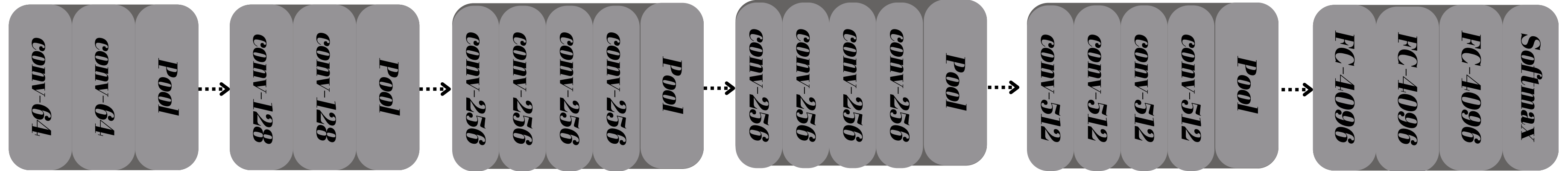
²VGG

****Karen Simonyan, Andrew Zisserman.2015.Very Deep Convolutional Networks for Large-Scale Image Recognition***

VGG-16



VGG-19



Accuracy
11 layer < 16 layer < 19 layer

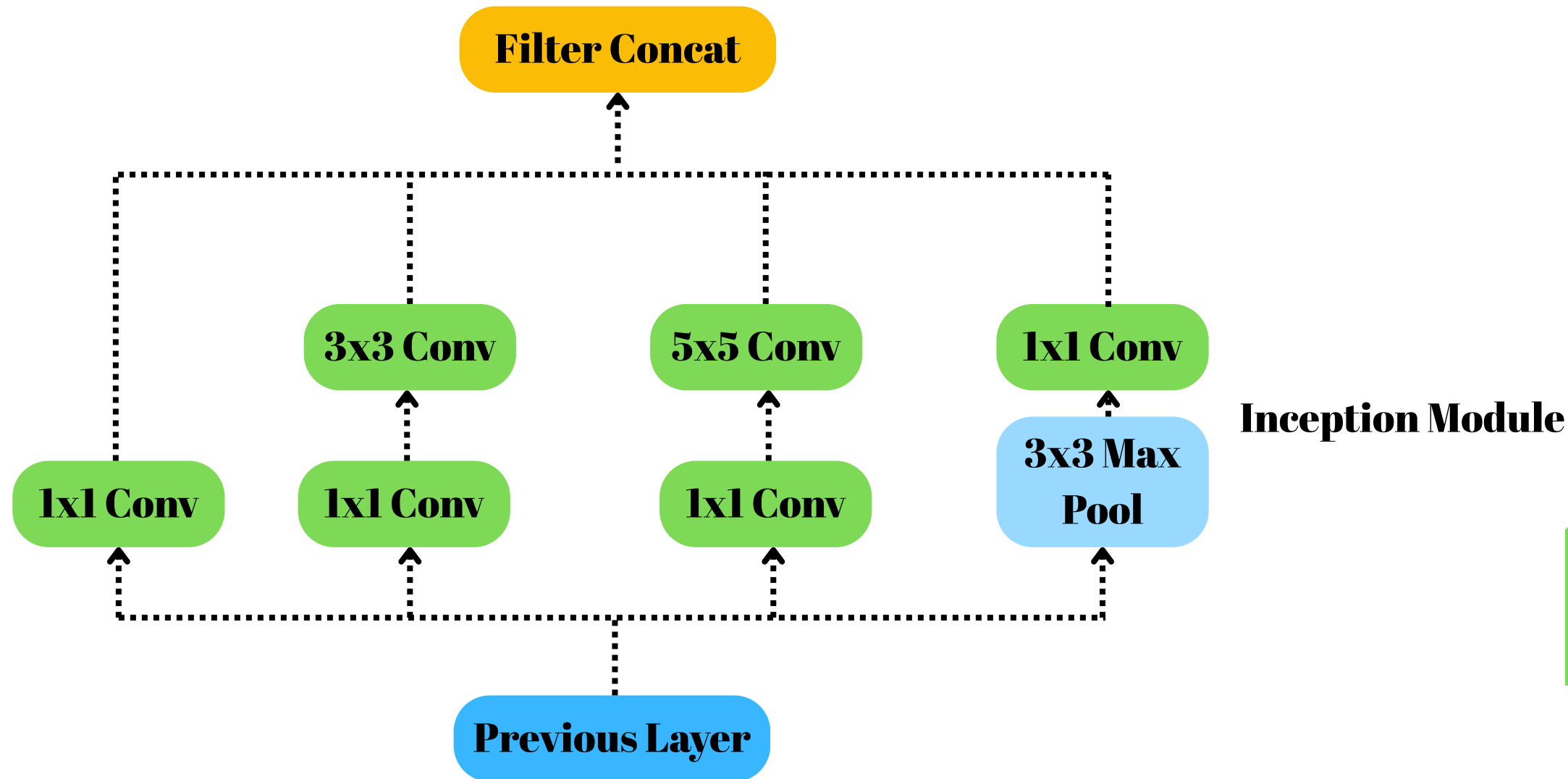
Inception

***Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke, Andrew Rabinovich. 2014.**

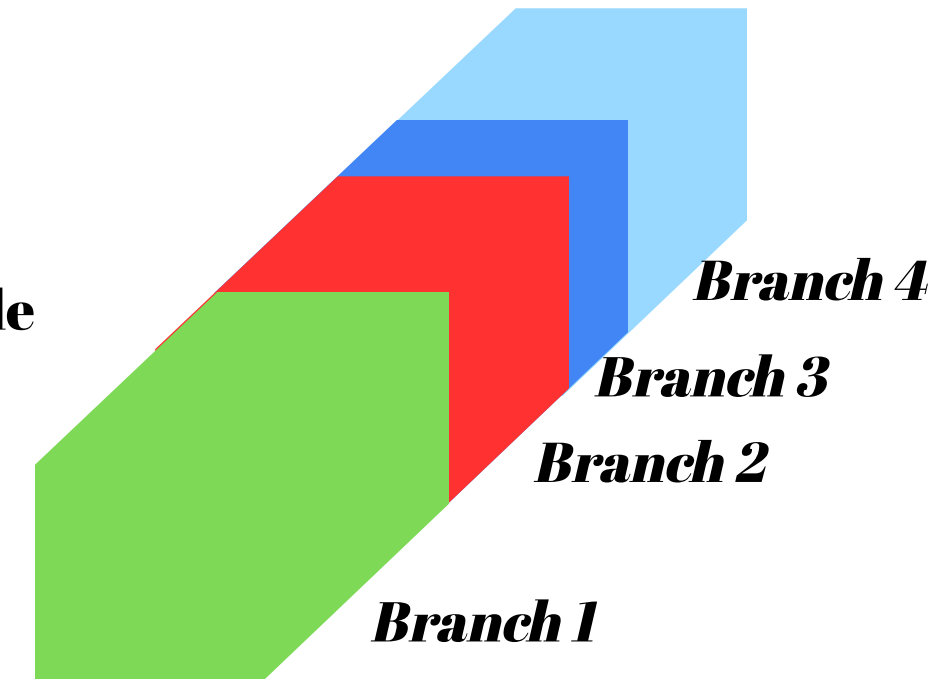
Going Deeper with Convolutions

GoogLeNet

- Farklı modeller hem depth hem de width artırarak modelin performansını artırmaya çalışıyor. Bu da daha fazla FLOP ve overfitting'e daha fazla yatkınlık demek oluyor.
- Conv uygularken hangi boyutu seçicem? 3x3 or 5x5
- Inception'ın amacı Hesaplama maliyetini düşürmek ve çok ölçekli feature extraction yapmak.



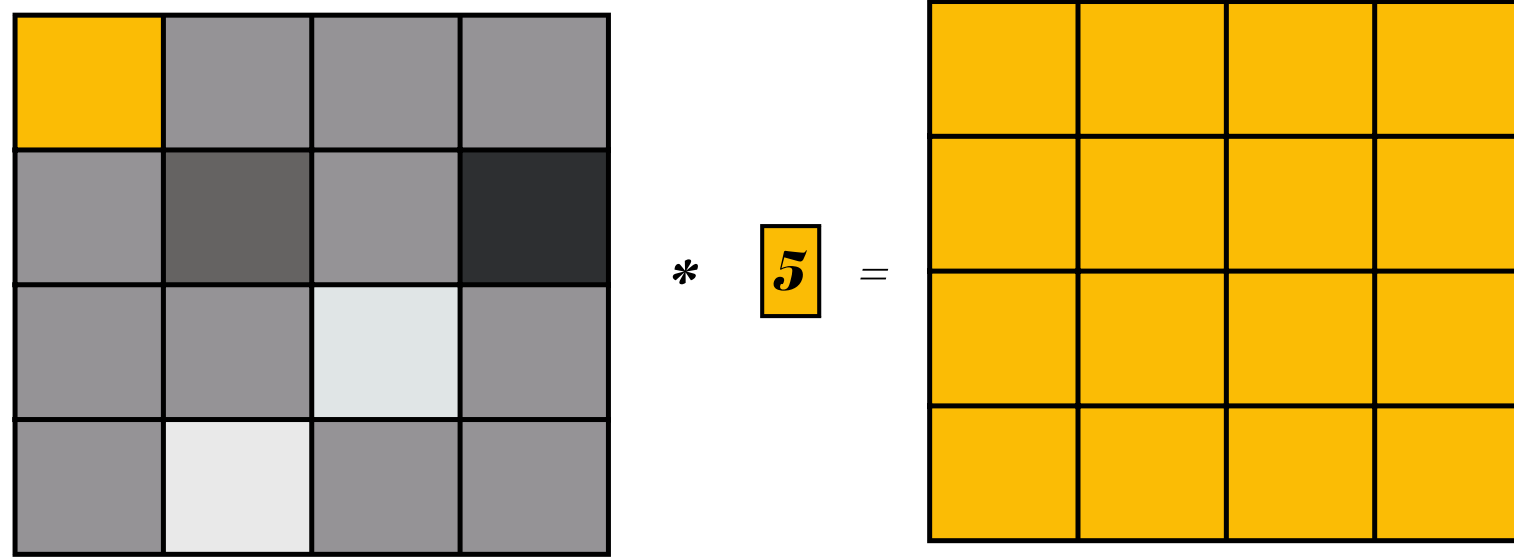
- Mimari tasarımı yapılırken her branch için spatial boyut aynı olacak şekilde tasarlanır.



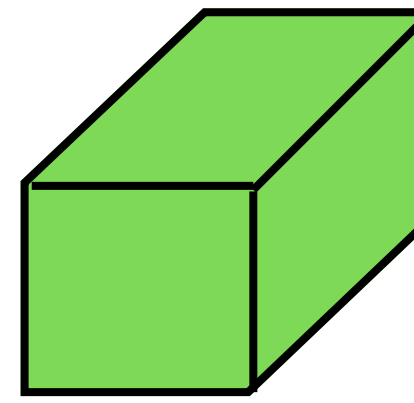
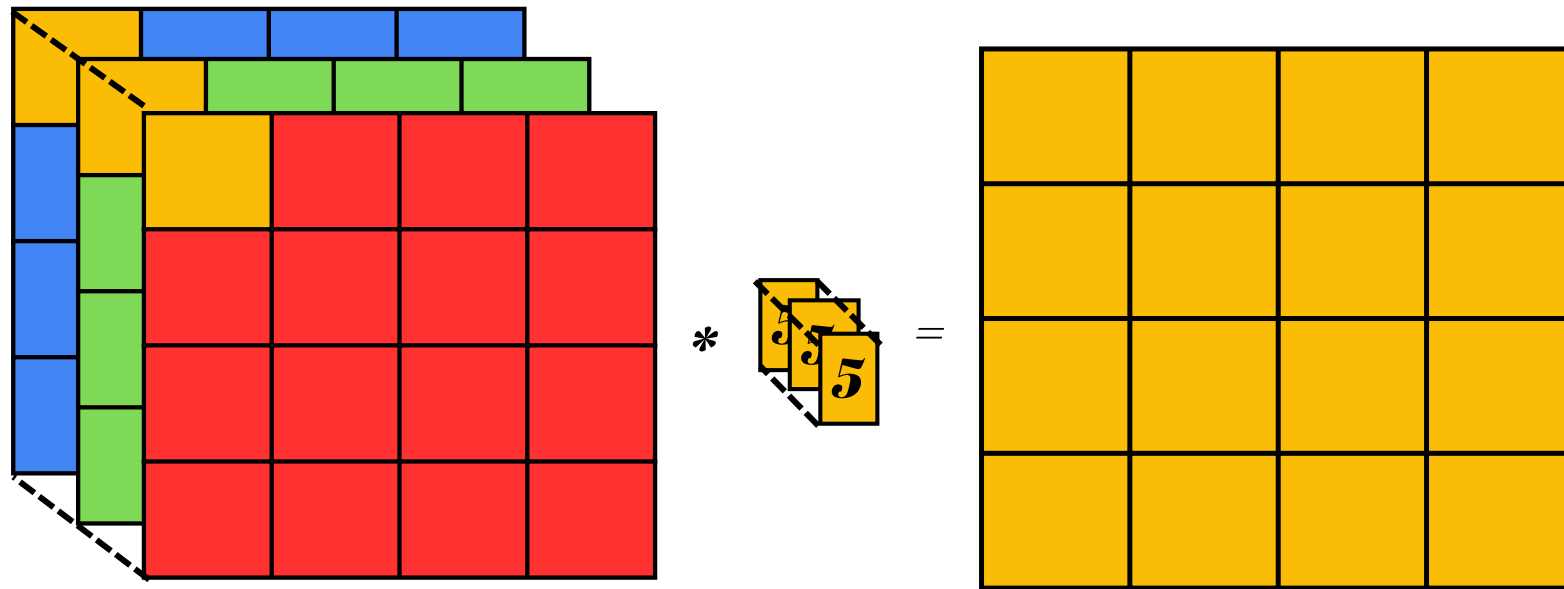
Inception²

***Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke, Andrew Rabinovich. 2014.**

Going Deeper with Convolutions

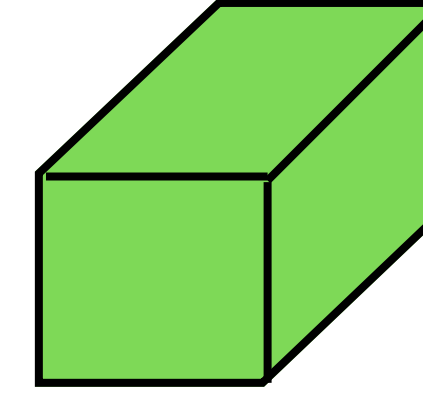


1x1 Conv gri skala üzerinde scalar gibi görünse de birden fazla filtre üzerinde uygulanınca dimension reduction etkisine sahiptir.



14x14x56

.....→
3x3 conv
same
128 filter

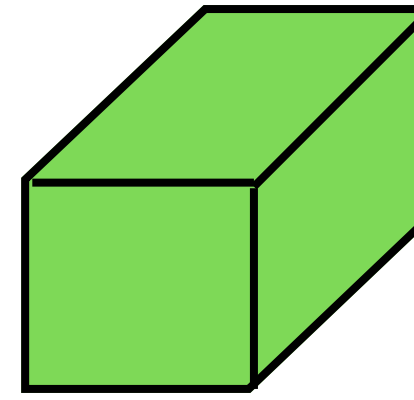


14x14x128

1 piksel için → 504

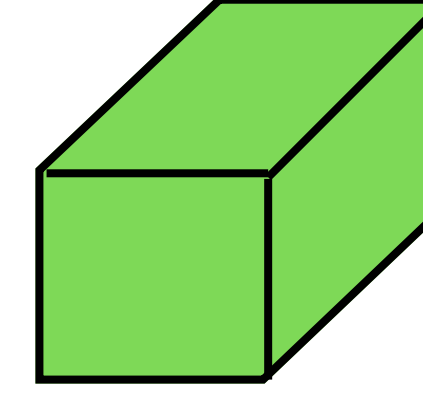
1 filtre için → 504 * 14 * 14 = 98784

128 filtre için → 504 * 14 * 14 * 128 = 12644352



14x14x56

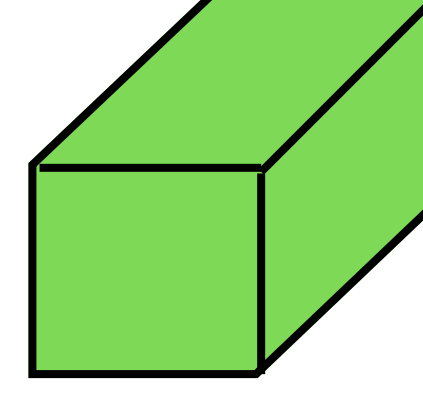
.....→
1x1 conv
same
16 filter



14x14x16

1 * 14 * 14 * 56 * 16
175616

.....→
3x3 conv
same
128 filter



14x14x128

3 * 3 * 14 * 14 * 16 * 128
3612672

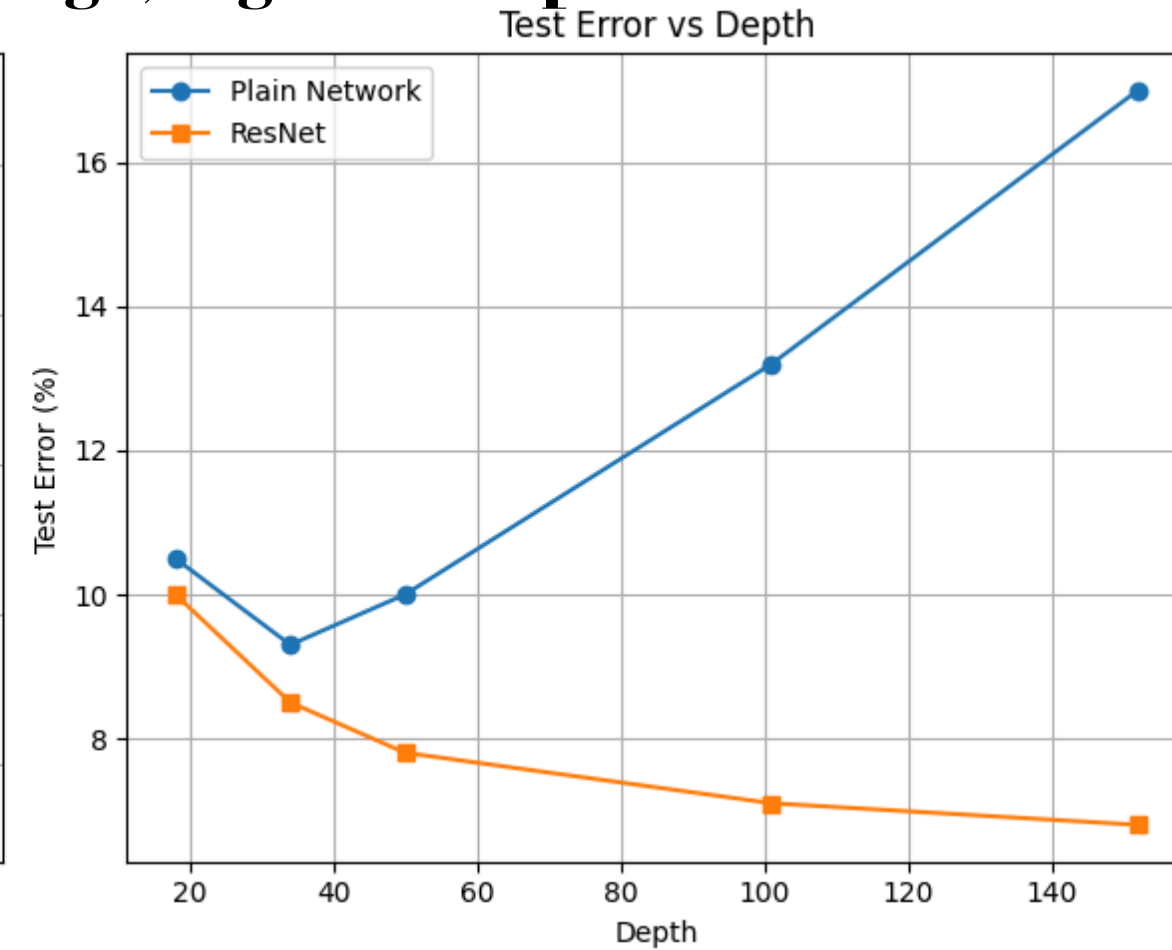
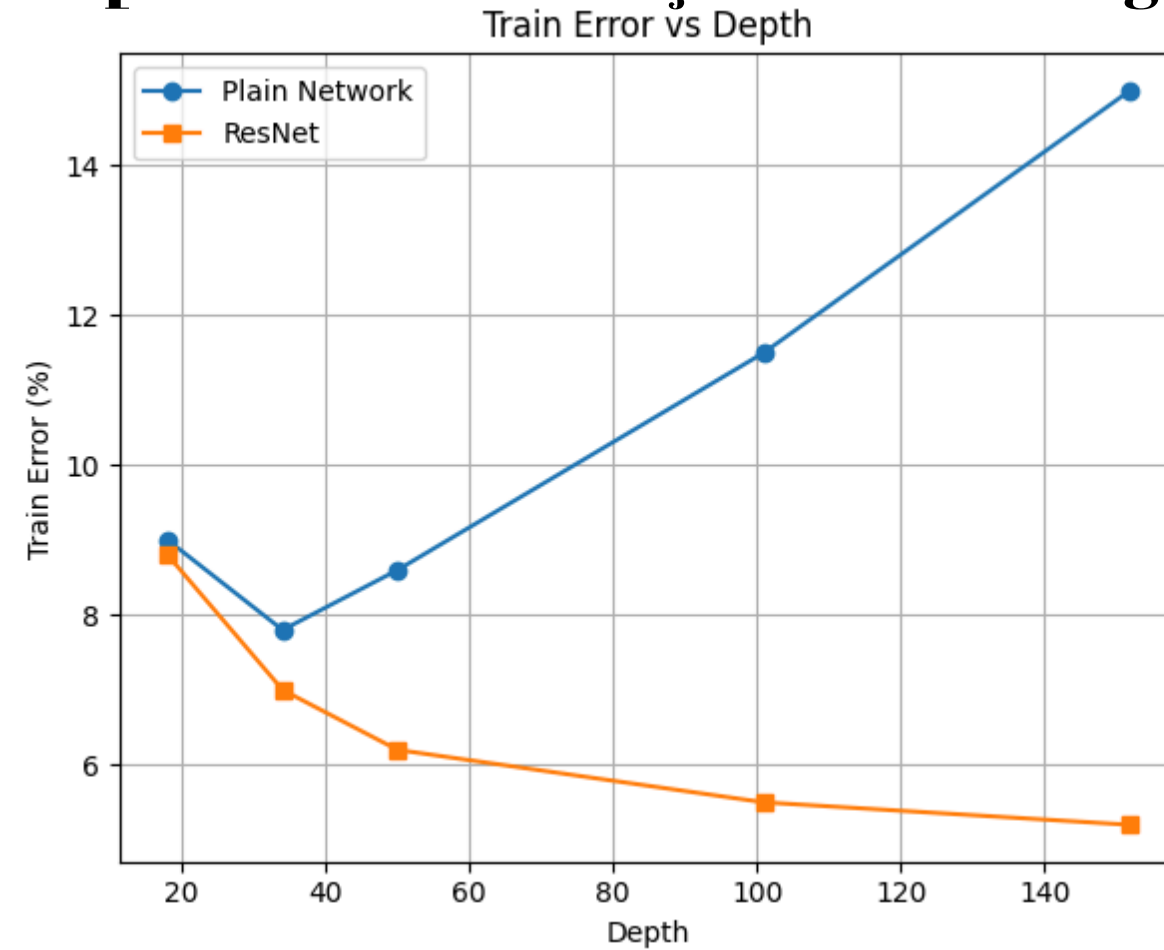
3788288

***Min Lin, Qiang Chen, Shuicheng Yan. 2013. Network In Network**

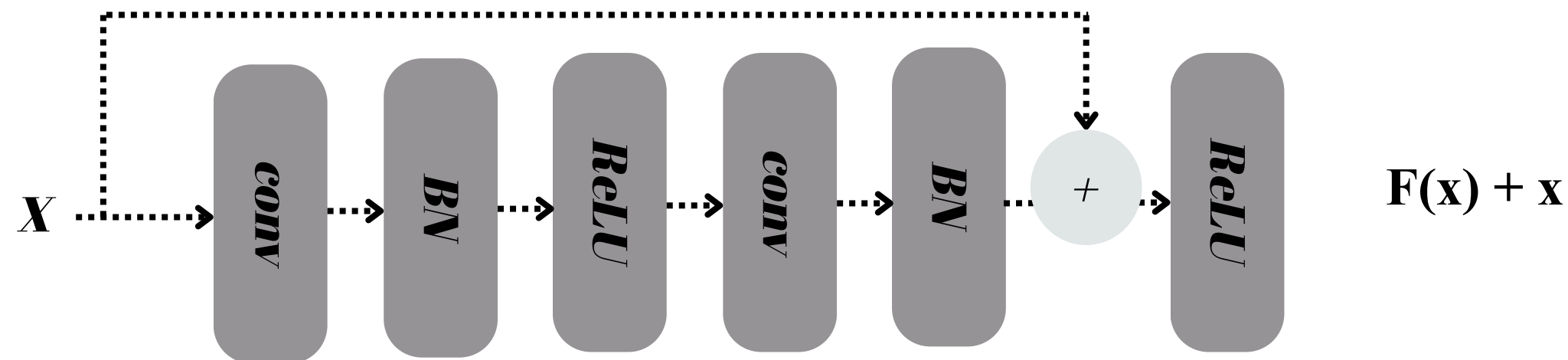
ResNet

**Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun. 2015. Deep Residual Learning for Image Recognition*

- Teoride CNN derinliği arttıkça performansın artması lazım ancak pratikte öyle olmuyor. ResNet derinlik artsa bile stabilize sağlayabilecek bir yöntem öneriyor.
- Derinlik arttıkça model performansının düşmesi overfitting değil, degradation problem olarak isimlendirilir



Residual Block:



²ResNet

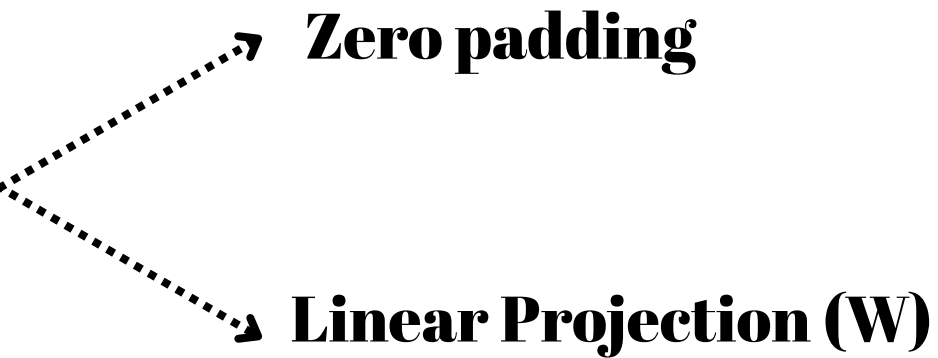
**Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun. 2015. Deep Residual Learning for Image Recognition*

Residual Block

- Gradient bypass sağlar. Bacpropagation sırasında residual connection üzerinden doğrudan atlayabilir.
- Derinlik bottleneck oluşturmaz.

Eğer X ve $F(X)$ aynı boyutta ise→ Element-wise addition

Eğer X ve $F(X)$ aynı boyutta değil ise



Zero padding

Linear Projection (W)

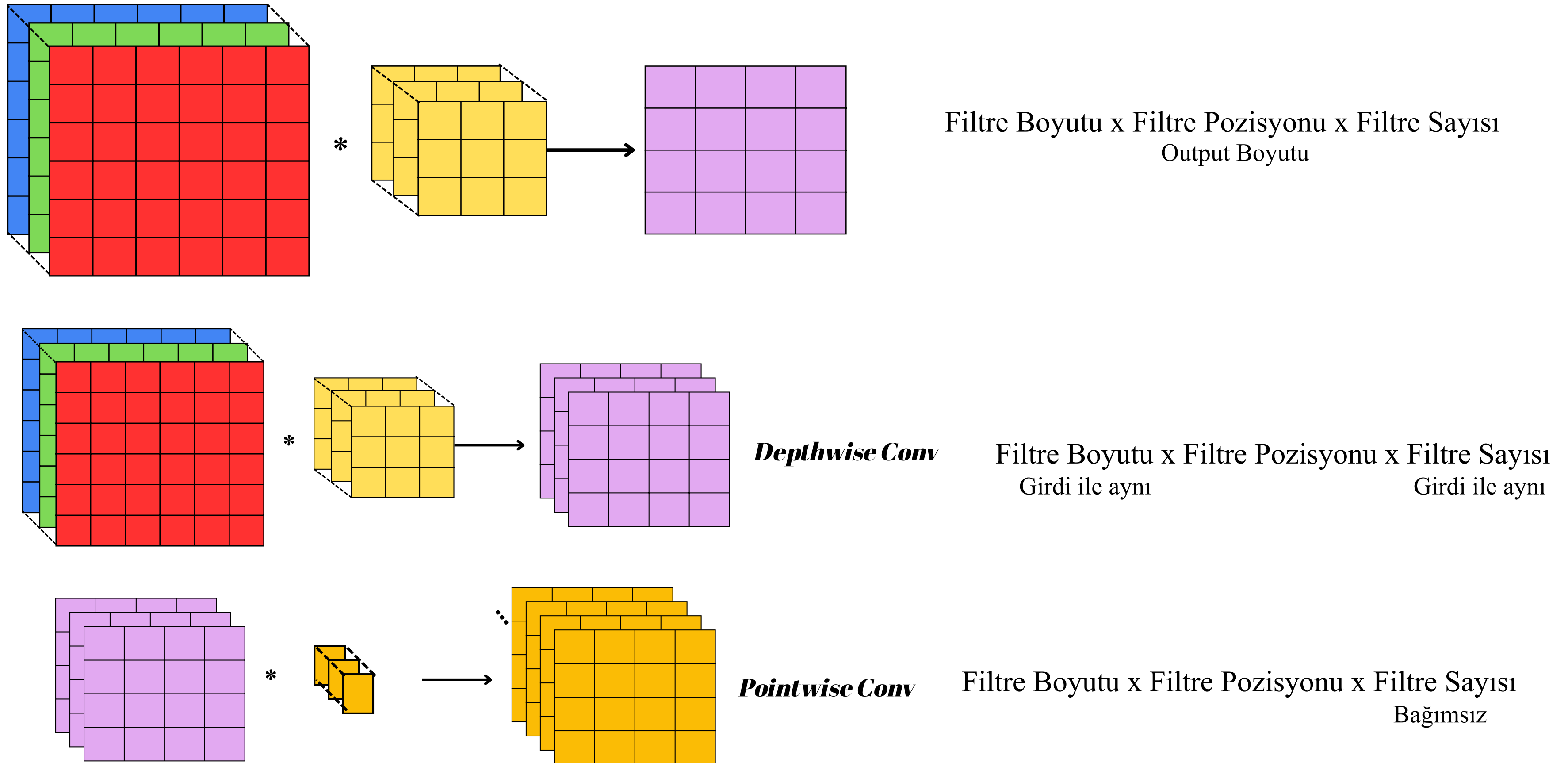
Key Points

- ResNet uygulamasının CNN dışında kullanılabileceğini de vurguluyor. (Kullanılıyor)
- VGG Paper'ı üzerine kurguludur. Onun gündeme getirdiği bir hipotezi tartışır.
- Residual Connection extra parametre tanıtmaz. Model büyüklüğünü artırmaz. Yük oluşturmaz.

MobileNet

****Andrew G. Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, Hartwig Adam. 2017.***

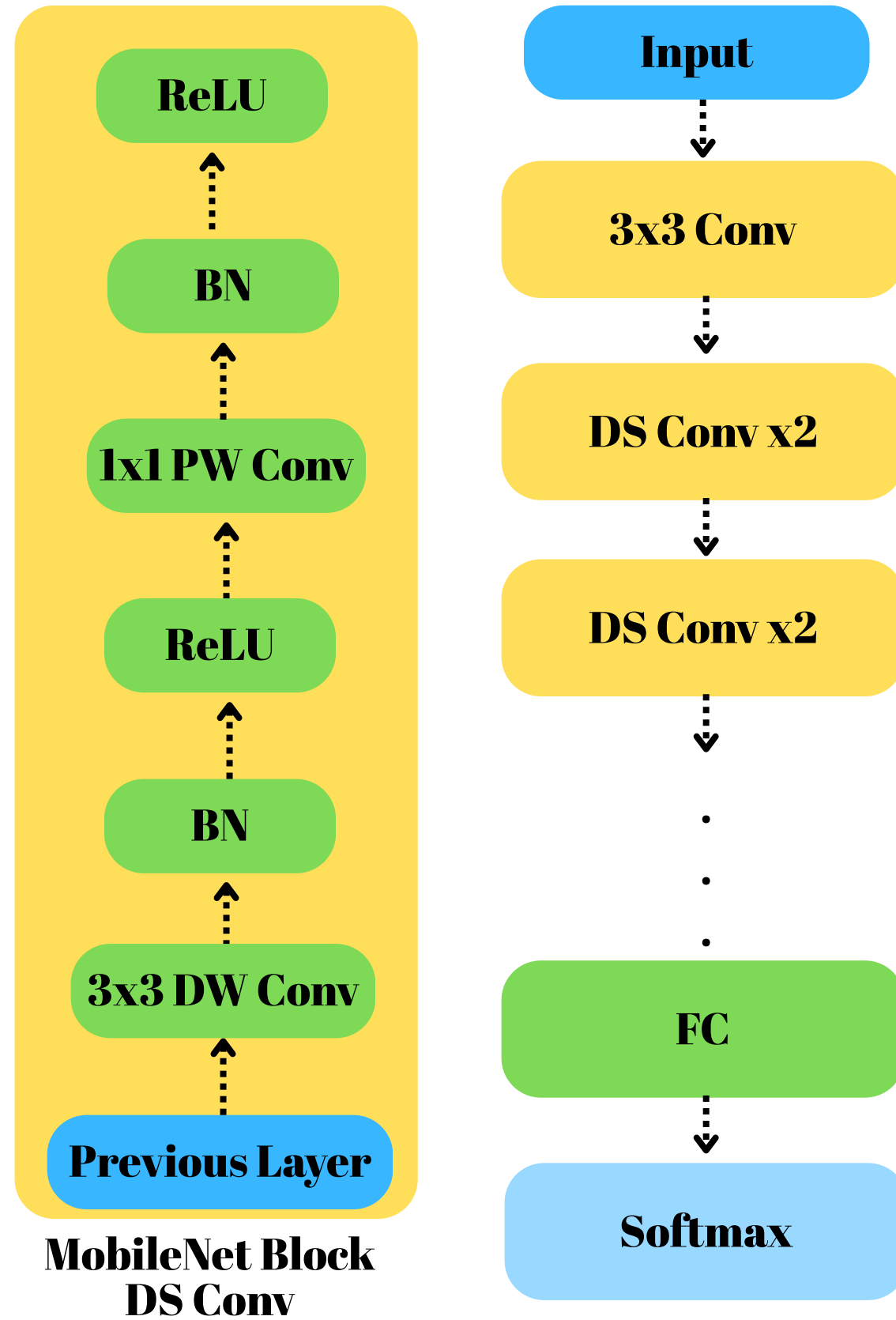
MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications



²MobileNet

****Andrew G. Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, Hartwig Adam. 2017.***

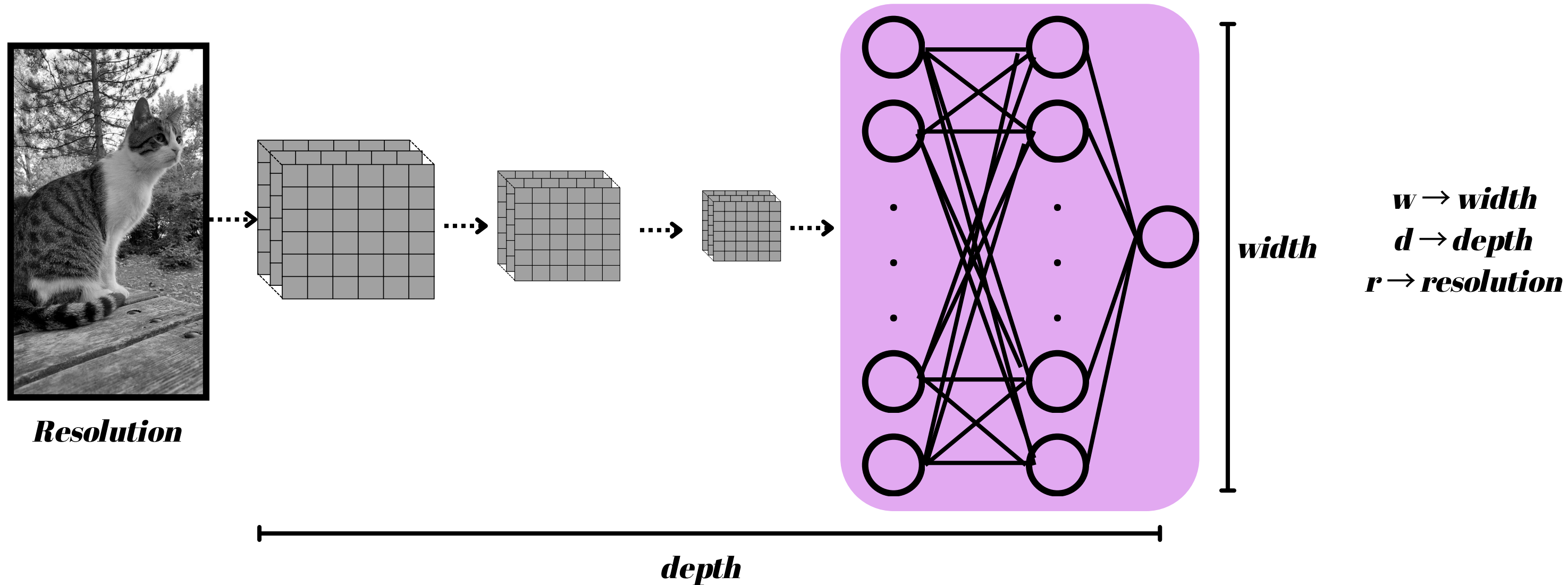
MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications



- **MobileNet Mobil cihazlar gibi sistem performansı yüksek olmayan cihazlarda (edge computing) çalışabilecek şekilde light bir model arayışının sonucunda ortaya çıkmıştır.**
- **Kullandığı metodlar ile birlikte parametre ve computation sayısı düşürmüştür.**

EfficientNet

**Mingxing Tan, Quoc V. Le. 2019. EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks*



$$\text{depth} = \alpha^\phi$$

$$\text{width} = \beta^\phi$$

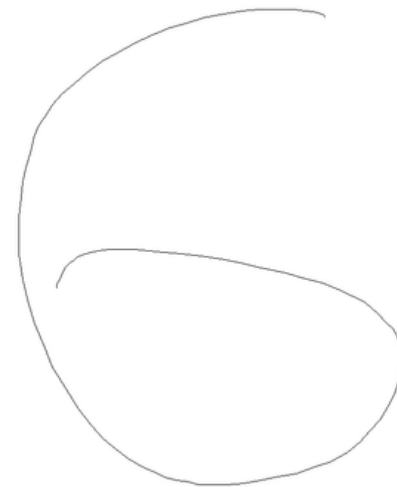
$$\text{resolution} = \gamma^\phi$$

$$\alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$$

- Model performansını artırmak için w,d,r değerlerinin hepsi ile oynanabilir.
- EfficientNet bu parametreler arasında matematiksel bir ilişki kurar ve hepsinin belirli bir ölçüde artırılması gerektiğini söyler.
- Bunun için Baseline bir model ile başlayıp (B0) en ağır modele kadar (B7) modeller inşa etmişler.

Data Augmentation - Motive

- Veri sayısının az olduğu durumda sayıyı artırmak için kullanılır.
- Overfitting riskini azaltır.
- Genelleme yeteneğini artırır.
- Stochastic veya Deterministic olabilir.
- Label preserving olmalıdır.

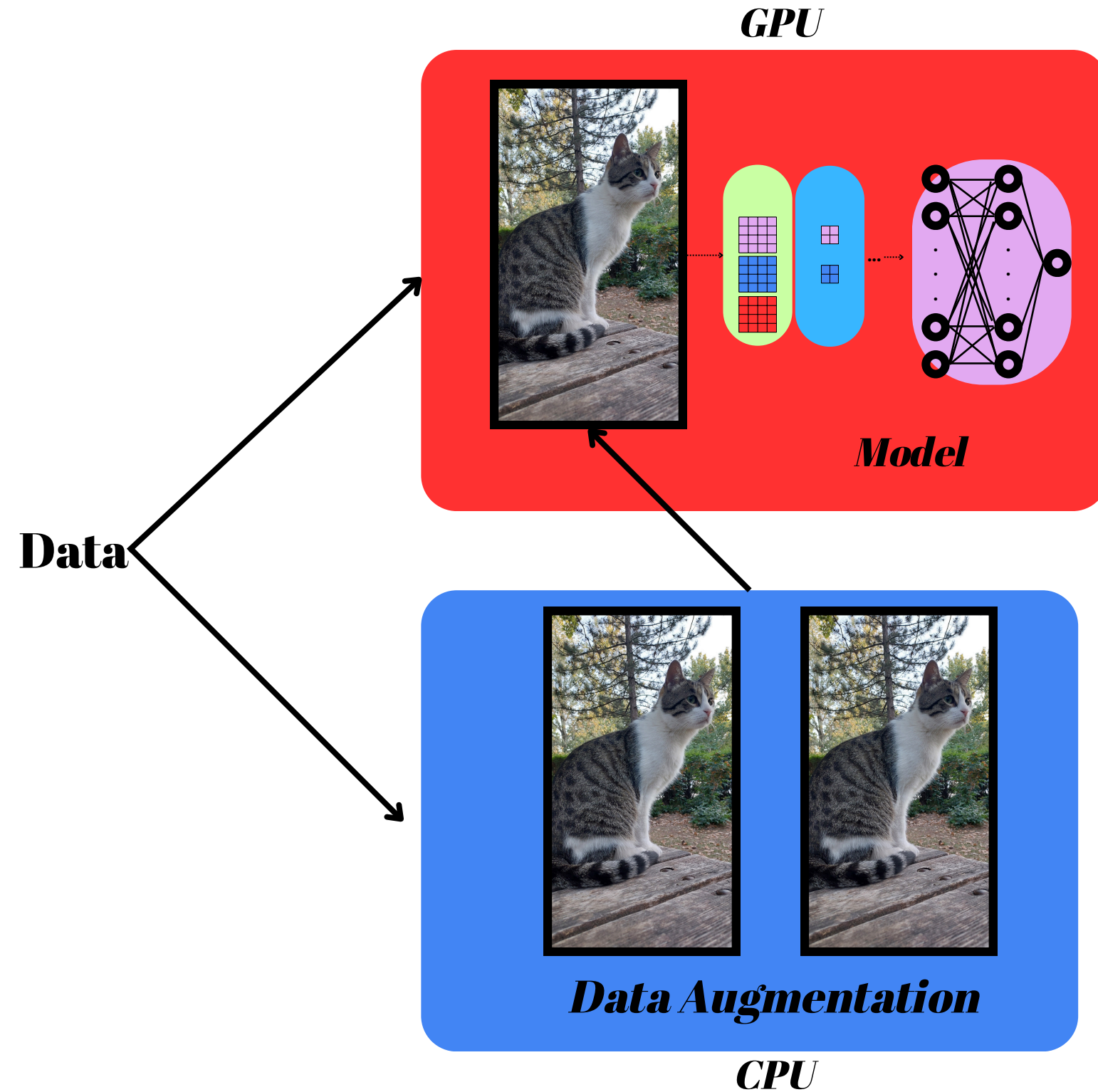


y_true = 6



y_true = 9

Data Augmentation - Pipeline



- GPU model eğitimi yaparken cpu üzerinde data augmentation uygulanır. Böylece bottleneck yaşanmaz, eğitim süresinde bir artış olmaz.
- Data sadece eğitim sırasında çoğaltılabilir. Örneğin 1000 adet resim 10000 adete çıkarılacaksa:
 - Önce 10000 resim üretilip depolanabilir (HDD/SSD)
 - Eğitim sırasında paralelde oluşturulabilir.

Albumentations

Compose [

A.RGBShift(p=0.3),→ %30 olasılıkla gerçekleşecek

A.VerticalFlip(p=0.3),

A.OneOf([

A.GaussNoise(p=1.0),

A.ISONoise(p=1.0),

A.MultiplicativeNoise(p=1.0)

], p=0.7)

]

3'ünden birinin uygulanma $p = 0.7$

Seçilenin uygulanma $p = 1.0$

Data Augmentation-Geometric

HorizontalFlip



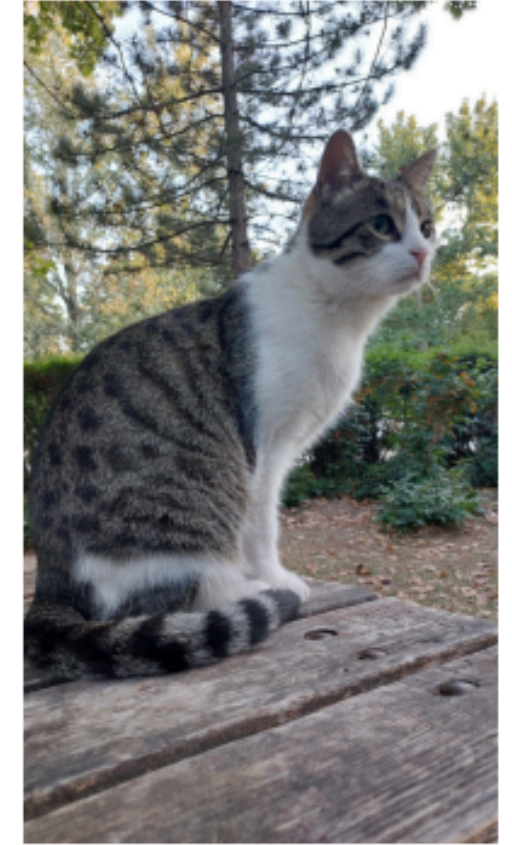
VerticalFlip



Rotate



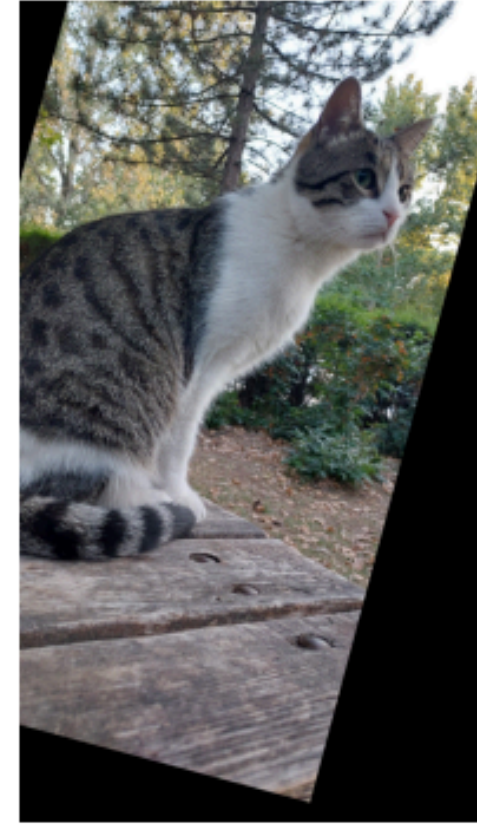
Original



RandomScale



ShiftScaleRotate



RandomCrop



Original



**süreyya hanım*

Data Augmentation

BrightnessContrast



HueSaturation



GaussNoise



GaussianBlur



RGBShift



CLAHE



RandomGamma



ChannelShuffle



İlerisi İçin

Object Detection

Image Segmentation

Image Preprocess

Medical Data

Video Process

Vision Transformer

5.1

5.2 ...

Elbet bir gün



- **Andrew NG tarafından verilen “Deep Learning” Kursunun CNN modülü. Kurs içerisinde bahsedilen birçok şey örtüşmektedir. Anlatım sade ve güzeldir.**

<https://learn.deeplearning.ai/specializations/deep-learning/lesson/xjdrt/computer-vision>

- **Stanford üniversitesi açık kaynak ders videoları. “Deep Learning for Computer Vision” isimli oynatma listesi. İçerisinde ilk 4 video Deep Learning kısmına odaklanıyor ve yer yer farklı mimarilerden bahsediliyor. Genel akış olarak Module-5 ile doğrudan örtüştüğünü söyleyemem ancak aradan seçmece video olarak veya bütün olarak izleyebilirsiniz.**

<https://www.youtube.com/playlist?list=PLoROMvodv4rOmsNzYBMe0gJY2XS8AQg16>

- **OpenCV University tarafından verilen ücretsiz kurs. İçerisinde OpenCV ile birlikte computer vision da anlatılıyor. OpenCV yeteneklerini geliştirmek isteyenler bakabilirler.**

<https://opencv.org/university/free-opencv-course/>